

Руководство по выживанию — TLS/SSL и сертификаты SSL (X.509)

 pro-ldap.ru/tr/zytrax/tech/ssl.html

Это руководство по выживанию рассматривает такую вводящую в ступор тему, как сертификаты TLS/SSL и X.509 (SSL), в том числе самоподписанные сертификаты. В нём присутствуют элементы того, что часто называют инфраструктурой открытых ключей (Public Key Infrastructure, PKI).

То, что мы в просторечии называем сертификатами SSL, технически является сертификатами X.509. Термин "сертификат SSL" получил распространение в связи с адаптацией Netscape формата сертификата X.509 (одного из [стандартов служб каталогов X.500 ITU](#)) при разработке этой компанией оригинальных версий протокола SSL (Secure Socket Layer) в те доисторические времена, когда мир был ещё молод, по планете бродили динозавры, а Интернет был дружелюбным местом. Термин "сертификат SSL" сохраняется и, скорее всего, сохранится в обозримом будущем, поскольку при произношении "сертификат SSL" и "сертификат X.509" первую из фраз выговаривать удобнее. Безусловно, эксперты-лингвисты смогут замечательно это обосновать, ссылаясь на то, что S звучит куда лиричнее, чем X. Для нас, простых смертных, главное преимущество первой фразы в том, что она короче (быстрее читать и произносить).

В настоящий момент руководство включает SSL, TLS, некоторые детали о X.509 и их применении, а также кое-какие разъяснения о типах сертификатов, в том числе о сертификатах EV, и о процессе доверия. Создание [самоподписанных сертификатов](#) представлено в виде рабочего примера использования пакета [OpenSSL](#). Также представлена [некоторая информация о содержимом различных типов файлов](#) (.pem, .p12, .pfx, .der, .cer), ключевые слова PEM и список сопоставления PKCS с RFC.

Вы можете либо купить сертификат SSL (X.509), либо сгенерировать свой собственный (самоподписанный) для тестирования или, в зависимости от приложения, даже применения в производственной среде. **Хорошие новости:** При использовании самоподписанных сертификатов Вы сэкономите несметное количество денег. **Плохие новости:** При использовании самоподписанных сертификатов никто кроме Вас и (возможно) Ваших ближайших родственников не сможет им доверять. Но, прежде чем выкладывать все свои сбережения за новенький блестящий сертификат X.509 (SSL) или даже за более дорогой сертификат EV SSL (X.509), Вам, вероятно, хотелось бы знать, что они делают и как они это делают. И если Вы впадаете в ступор как только люди начинают говорить об SSL, защите информации и сертификатах — самое время туда впасть. Это руководство не будет лёгким чтивом.

О вездесущих SSL и ACME: Использование TLS (вернее, наличие сертификата, используемого в TLS) сегодня стало де-факто стандартом аутентификации веб-сайтов (хотя, на наш взгляд, DNSSEC — бесконечно лучшее решение, но, увы, оказавшееся склонным к суициду). И действительно, основные поисковые системы теперь рассматривают отсутствие HTTPS (HTTP поверх TLS) как повод для снижения позиций результатов поиска (это означает, что результаты поиска отображаются на основании некоторой политики, а не только на основании содержимого ресурса). Однако, у этого подхода есть ряд побочных эффектов, а именно: нужно иметь (купить) сертификат SSL; "закон больших чисел" гласит, что даже крошечная ошибка в конфигурации приведёт к достаточно большому количеству предупреждений безопасности (покрасневшие строки состояния и т. д.), так что всё предприятие рискует завалиться под своим собственным весом. К хорошим новостям можно отнести появление [Let's Encrypt](#) и Automatic Certificate Management Environment (ACME, Среды автоматического управления сертификатами). Let's Encrypt предоставляет бесплатные сертификаты со сроком действия 90 дней, а также автоматизирует для ряда популярных платформ процессы первоначальной установки и последующего перевыпуска сертификатов посредством ACME или, в случае возникновения проблем, выполнения всего этого вручную. Первый стандарт, основанный на их работе, — [RFC 8555](#), в котором описаны процессы автоматической проверки факта владения доменом, а также выпуска (и отзыва) сертификатов.

Примечание: Основной репозиторий RFC поддерживается [IETF](#), текстовые версии (которые считаются нормативными), а также PDF и HTML-версии можно найти на www.ietf.org/rfc/rfcXXXX.txt или на www.rfc-editor.org/rfc/rfcXXXX.txt (где XXXX — состоящий из 4 цифр номер RFC, при необходимости дополненный слева нулями). Опубликованные на настоящий момент RFC размещены на <https://www.rfc-editor.org/info/rfcXXXX>, здесь содержится различная информация и ссылки на текстовые (нормативные), а также PDF и HTML (ненормативные) версии документов. RFC в также можно посмотреть на <http://datatracker.ietf.org/doc/rfcXXXX/>, здесь же содержится различная статусная информация по RFC (в том числе сообщения об ошибках), а также ссылки на просмотр в различных форматах, таких как текст, PDF и HTML (это версии документов, находящиеся в работе). Наконец, существует [страница поиска по RFC](#).

<укоренившиеся привычки> В данном документе гиперссылки на RFC указывают на текстовые (нормативные) версии, которые мы скопировали на наш сайт, когда эти RFC были выпущены. Мы начинали делать это очень, очень давно, когда мир был еще молод и RFC содержались в каких-то странных местах, иногда меняющих своё местоположение, а производительность и доступность этих репозиторий оставляла желать лучшего (мягко сказано). Сегодня всё это далеко не так. Репозиторий RFC — надежный и быстрый сервис. Тем не менее, мы упорно продолжаем следовать нашим укоренившимся привычкам без всякой видимой причины (старого пса новым

трюкам не выучишь...). Если Вы хотите/предпочитаете/нуждаетесь в более продвинутом представлении RFC, рекомендуем Вам использовать одну из указанных выше ссылок. Если же Вам просто нужно почитать RFC, не стесняйтесь нажимать на наши ссылки. </укоренившиеся привычки>

Примечание переводчика: в русском варианте страницы ссылки указывают на HTML-версии RFC на сайте <https://tools.ietf.org/html/>.

Время от времени, когда нам больше нечем занять свою жизнь, мы обновляем эту страницу и ведём [журнал изменений](#) на тот случай, если кто-то захочет прочитать её ещё разок. Конечно, к Вам это не относится, ведь Вы всё отлично схватываете с первого раза, верно?

Содержание:

- [Обзор протокола TLS/SSL](#)
- [TLS/SSL 1.3 — детальное описание](#)
- [Последовательность обмена сообщениями протокола TLS/SSL 1.3](#)
- [TLS/SSL 1.2 \(и 1.1\) — детальное описание](#)
- [Последовательность обмена сообщениями протокола TLS/SSL 1.2](#)
- [Шифрование — обзор](#)
- [Обзор сертификата X.509](#)
- [Типы и терминология сертификатов X.509](#)
- [Цепочки сертификатов X.509](#)
- [Использование сертификатов X.509 \(Ну и зачем я это купил?\)](#)
- [Сертификаты X.509 в организациях, предоставляющих услуги веб-хостинга](#)
- [Протоколы, связанные с сертификатами \(CMP, CMC/CMS, SCVP, OCSP, HTTP\)](#)
 - [Протокол онлайн-получения статуса сертификата \(OCSP\)](#)
- [Формат сертификата X.509](#)
 - [Замечания по атрибутам сертификата subject и subjectAltName \(SAN\)](#)
- [Списки отзыва сертификатов X.509 \(CRL\)](#)
- [Процесс доверия — удостоверяющие центры и сертификаты X.509](#)
- [Сертификаты X.509 расширенной валидации \(EV\)](#)
- [Примеры — создание самоподписанных сертификатов \(OpenSSL, несколько методов\)](#)

- [Замечания по форматам файлов, связанных с TLS/SSL](#)
 1. [Формат PEM](#)
 2. [Ключевые слова \(метки\) в строках BEGIN PEM-файла](#)
 3. [Расширения \(суффиксы\) файлов](#)
 4. [Связки сертификатов](#)
 5. [Команды OpenSSL для конвертации, извлечения и манипуляций с сертификатами и ключами](#)
 6. [Таблица соответствия стандартов PKCS и RFC](#)
 7. [Работа с сертификатами в основных браузерах](#)
- [RFC по теме](#)
- [Руководство по выживанию в ASN.1 и DER](#)
- [История изменений этой страницы](#)

Протокол TLS/SSL

Основное предназначение сертификатов SSL (X.509) — использование совместно с протоколом TLS/SSL. Secure Sockets Layer (SSL, уровень защищённых сокетов) — протокол, изначально разработанный Netscape в 1992 году для безопасного обмена информацией между web-сервером и браузером по незащищённым сетям.

Претерпел несколько изменений, нынешняя [версия 3](#) датируется 1995 годом и используется в различных клиент-серверных приложениях. В связи с распадом Netscape спецификации SSL в дальнейшем обновляться не будут. Таким образом, этот стандарт повторил судьбу известного [мёртвого попугая](#), и, в конце концов, в [RFC 7568](#) SSL v3 признан устаревшим. Теперь это официально мёртвый стандарт, и потому впредь его использовать не стоит, хотя бы ради сохранения на Земле главенства разумного, доброго и вечного.

IETF стандартизировала Transport Layer Security (TLS, безопасность транспортного уровня) версии 1, незначительно отличающийся от SSL, в [RFC 2246](#), версии 1.1 в [RFC 4346](#), версии 1.2 в [RFC 5246](#), наконец, версии 1.3 (в которой представлено существенное изменение в протоколе рукопожатия) в [RFC 8446](#). Кроме того, в [RFC 3546](#) определено несколько расширений для случаев, когда TLS используются в системах с ограниченной пропускной способностью, таких как беспроводные сети, в [RFC 6066](#) определён ряд расширений TLS, внесённых в формат расширенного приветствия клиента (представленного в TLS 1.2), в [RFC 6961](#) определён метод уменьшения трафика, когда клиент запрашивает у сервера информацию о состоянии сертификата. И, наконец, в [RFC 7925](#) определяется, что происходит с TLS (и DTLS) при его использовании в IoT (Internet of Things, Интернете вещей или Интернете вещичек, как презрительно называем его мы).

При первоначальной установке защищённого соединения, в зависимости от реализации, будут проводиться переговоры о поддержке конкретного протокола из

набора SSLv3, TLSv1, TLSv1.1, TLSv1.2 или TLSv1.3. Все так привыкли к названию SSL, что в большинстве случаев то, что называют SSL, скорее всего является использованием TLS.

Примечание: SSLv2 был отменён (официально получил статус исторического) документом [RFC 6176](#), в котором приводится ужасающий список всех его недостатков. Та же печальная участь теперь постигла и SSLv3: он был отменён [RFC 7568](#). Все встречающиеся далее упоминания SSL сохранены в тексте по причине широкого распространения этого термина (он до сих пор используется очень часто), однако читателям следует помнить, что речь идёт о TLS.

Область действия TLS/SSL

Следующие определения взяты из [раздела 1 RFC 8446](#) (но относятся и ко всем предыдущим версиям):

Аутентификация: Серверная сторона канала всегда аутентифицирована; клиентская сторона может быть аутентифицирована. **Конфиденциальность:** Данные, передаваемые по каналу после его установления, видны только конечным точкам. **Целостность:** Данные, передаваемые по каналу после его установления, не могут быть изменены злоумышленниками без обнаружения этого факта.

(Текст был переформатирован, некоторые его фрагменты были для краткости опущены.)

Замечание о целостности: TLS/SSL является протоколом коммуникации. TLS/SSL гарантирует целостность данных, предоставленных некоторым приложением в программное обеспечение протокола TLS/SSL с использованием *программного интерфейса приложения* (API) (не определён в стандартах TLS/SSL). Приложение может получить эти данные различными путями, например, сгенерировать с помощью некоего алгоритма, считать из области RAM, файловой системы, удалённого источника, и т.д. Целостность этих исходных данных не обеспечивается протоколом TLS/SSL. То есть, можно сказать, что протокол TLS/SSL обеспечивает целостность передачи данных, а не целостность содержимого. Так, например, если веб-сайт был **повреждён** вследствие некоторой активности файловой системы, то такое **повреждение** не будет и не может быть выявлено протоколом TLS/SSL. Есть одно, но важное, исключение. В ходе протокола рукопожатия сервер TLS/SSL обычно предоставляет сертификат (клиент также может опционально предоставить сертификат). Целостность содержимого этого сертификата гарантируется, но не протоколом TLS/SSL, а определяемым отдельно процессом валидации сертификата.

Операции TLS/SSL

TLS/SSL работают поверх TCP, но ниже по уровню тех конечных пользовательских протоколов, которые они защищают, таких как HTTP или IMAP (как показано на рисунке 1):

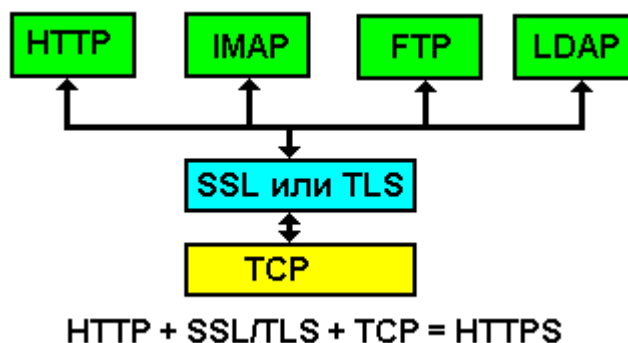


Рисунок 1 — Уровень TLS/SSL

За самими протоколами TLS/SSL не закреплено какого-либо общеизвестного номера порта — вместо этого при использовании с протоколом более высокого уровня, например HTTP, принято обозначать, что этот протокол будет использовать защищённый вариант (HTTPS в случае HTTP), у которого как раз есть общеизвестный номер порта (или порт по умолчанию). Обозначение HTTPS просто указывает на то, что нормальный протокол HTTP будет работать поверх соединения TLS/SSL, которые в свою очередь работают поверх TCP. В случае HTTPS общеизвестный номер порта — 443, в случае IMAPS — 993, POP3S — 995 и так далее.

Дальнейшее описание требует некоторого знакомства с терминами MAC (Message Authentication Code, аутентификационный код сообщения), односторонние хэши, симметричные и асимметричные криптографические алгоритмы (а для TLS 1.2 и выше — с алгоритмом распределения ключей Диффи-Хеллмана). Для тех, кто с ними не знаком, рекомендуется почитать [это руководство по выживанию, касающееся криптографии](#). Осторожно, прочтение этого материала может вызывать переутомление и фобии.

Примечания:

1. Родственный протокол, Datagram Transport Layer Security (DTLS, безопасность транспортного уровня для дейтаграмм), определяет сервис безопасности при использовании UDP ([RFC 6347](#), обновлён в [RFC 7507](#)). Поскольку его принципы аналогичны TLS (хотя и отличаются в некоторых деталях), в дальнейшем в данном руководстве DTLS не обсуждается.

2. Термин TLS 1.2 Suite B (определённый в [RFC 6460](#)) определяет набор шифров (смотрите ниже), совместимых с NSA Suite B Cryptography, и имеет значение только при использовании TLS для приложений в интересах национальной безопасности США.
3. Жизнь не стоит на месте. С выходом [RFC 8423](#) термин Suite B ушёл в прошлое, поскольку Агентство национальной безопасности США упразднило Suite B и заменило его на пакет Commercial National Security Algorithm Suite (CNSA Suite). Как и прежде, определённые в CNSA Suite алгоритмы имеют значение только при использовании TLS для приложений в интересах национальной безопасности США, вплоть до уровня секретности TOP SECRET.
4. Жизнь неумолимо движется вперёд. В [RFC 8446](#), с задержкой в несколько лет, был представлен TLS 1.3. TLS 1.3 содержит существенные изменения в [протоколе рукопожатия TLS](#), в том числе новый метод ведения переговоров при установлении сессии, повышающий количество зашифрованных сообщений и снижающий общее количество сообщений, используемых в рукопожатии. При определённых условиях TLS 1.3 не поддерживает обратную совместимость с TLS 1.2. Поэтому, если клиент или сервер поддерживает TLS 1.3, то другая сторона, поддерживающая TLS 1.2 (или TLS 1.1), может оказаться не в состоянии договориться об установлении сессии без обновления поддерживаемой версии протокола.

Обзор — установление защищенного соединения

При установлении защищенного соединения с помощью TLS/SSL, например при использовании HTTPS (порт по умолчанию — 443), между клиентом, который всегда является инициатором соединения, и сервером происходит обмен сообщениями. Первый набор сообщений называется протоколом рукопожатия (Handshake Protocol), после обмена этими сообщениями и клиент и сервер входят в протокол записи (или данных) (Record (Data) Protocol). При обмене сообщениями во время протокола рукопожатия достигаются следующие цели:

1. Устанавливается, какой вариант протокола из поддерживаемого (в зависимости от реализации) набора, — TLSv1, TLSv1.1, TLSv1.2, — будет использоваться. Всегда выбирается самый последний из возможных вариантов, то есть у TLSv1.1 всегда будет приоритет перед TLSv1 в случае, если и клиент и сервер поддерживают оба варианта. Клиент предлагает список вариантов, а сервер выбирает из предложенного списка.

2. Отправляются аутентификационные данные. Обычно сервер посылает аутентификационную информацию в форме сертификата (встроенную в сертификат) X.509 (SSL), но протокол поддерживает и другие методы.

Примечание: В [RFC 8492](#) (имеющем, правда, только статус INFORMATIONAL) описана система парольной аутентификации, которая избавляет от необходимости иметь сертификат.

3. Устанавливается идентификатор (ID) сессии, таким образом, при необходимости сессия может быть перезапущена.
4. Происходят переговоры о наборе шифров, состоящем из алгоритма обмена ключами вместе с типом алгоритма шифрования объёмных данных и типом [MAC](#), которые будут использоваться в последующей сессии обмена данными (протокол записи). Обычно в качестве алгоритма обмена ключами используется [асимметричный алгоритм \(с закрытым и открытым ключом\)](#), такой как RSA, DSA или ECC (Elliptic Curve Cipher, шифр на основе эллиптических кривых, смотрите [RFC 5289](#)). Асимметричные алгоритмы потребляют очень много ресурсов процессора и потому для последующего шифрования объёмных данных (протокол записи) используются [симметричные шифры](#). Алгоритмы обмена ключами применяются для передачи информации, на основании которой могут быть независимо вычислены сеансовые ключи для симметричного шифра. MAC используется для защиты целостности передаваемых/получаемых данных во время протокола записи.

Конечно, это упрощённая схема, и во время установления соединения может совершаться обмен и другими данными, например, в процессе так называемой взаимной аутентификации у клиента для его аутентификации может быть запрошен сертификат X.509 (SSL), но описанный нами процесс — это *наиболее распространённый* случай для любых версий SSL (в настоящий момент все они устарели) и для TLS.

Выбор читателя (дилемма): С появлением TLS версии 1.3 протокол TLS стал намного сложнее. Далее Вы можете ознакомиться либо с [детальным описанием TLS до версии 1.3 \(TLS 1.1 и TLS 1.2\)](#), либо с не менее [детальным описанием TLS 1.3 \(и, будем надеяться, всех последующих версий\)](#). Если у Вас высокий болевой порог, можете прочитать и то, и другое. И наоборот, если Вы печётесь о своём душевном здоровье, Вы вправе [пропустить это нагромождение малопонятных слов](#).

Детальное описание TLS 1.3



Рисунок 2 — Последовательность обмена сообщениями протокола TLS 1.3

Примечания:

1. В фазе протокола рукопожатия проводятся переговоры и устанавливается соединение, а в фазе протокола записи происходит передача (инкапсуляция) зашифрованного потока данных, таких как HTTP, SMTP или IMAP.
2. На рисунке 2 чёрными стрелками показаны сообщения, отправляемые открытым текстом (незашифрованные). Фиолетовыми — сообщения, отправляемые с использованием того алгоритма шифрования объёмных данных, о котором стороны договорились в процессе переговоров (при этом используется вычисленный общий секретный ключ, предназначенный для защиты данных процесса рукопожатия); целостность сообщений обеспечивается с помощью того алгоритма MAC, о котором стороны договорились в процессе переговоров. Зелёным цветом показаны сообщения, отправляемые с использованием того алгоритма шифрования объёмных данных, о котором стороны договорились в процессе переговоров (при этом используется вычисленный общий секретный ключ, предназначенный для защиты данных приложения); целостность сообщений обеспечивается с помощью того алгоритма MAC, о котором стороны договорились в процессе переговоров. Наконец, пунктирными стрелками (независимо от цвета) показаны опциональные сообщения, которые могут не присутствовать в обмене.

3. На рисунке 2 сообщения для ясности показаны как отдельные объекты, на практике они обычно объединяются в блоки.
4. Версии TLS/SSL, предшествующие TLS 1.3, позволяют договориться об использовании какого-либо алгоритма сжатия данных (как составной части набора шифров). С учётом скорости современных сетей сжатие данных использовалось всё реже или вообще не использовалось, и обычно значение выбранного алгоритма сжатия в согласованном наборе шифров устанавливалось в NULL (не используется). И действительно, сжимать данные стали настолько редко, что в TLS 1.3 эту опцию вообще убрали, и все вздохнули с облегчением.

TLS 1.3, полный протокол рукопожатия — детальное описание (рисунок 2)

Если говорить в общем, в TLS 1.3 уменьшено количество циклов приёма-передачи (round-trips, RTTs), требуемых для установления соединения, шифруется максимально возможное количество трафика, а также значительно снижено количество опций, что позволило усилить базовый уровень безопасности. Ключевым моментом, который необходимо отметить в полном протоколе рукопожатия, является то, что в первых двух сообщениях (ClientHello (1)/ServerHello (2)) устанавливаются только используемая версия и криптографический контекст. После того, как соединение с использованием полного протокола рукопожатия было установлено, последующие сессии между теми же клиентом и сервером могут быть [перезапущены \(или, на жаргоне TLS, возобновлены\)](#).

1. **ClientHello (1)**: Сообщение ClientHello TLS 1.3 содержит определённые фиксированные поля, многие из которых присутствуют в нём для поддержания обратной совместимости. Поддерживаемая версия/вариант протокола всегда установлена в TLS 1.2 (но сообщение должно включать расширение `supported_versions` со значением, установленным в TLS 1.3). Другие поля: поддерживаемые наборы шифров в порядке предпочтения ([приложение B.4 RFC 8446](#)), случайное значение (отметка текущего времени) в 32 октета (которое в дальнейшем используется в качестве материала для формирования ключа), идентификатор сессии (всегда 0 для обратной совместимости), метод сжатия (всегда 0= NULL для обратной совместимости). Кроме того, должны (в зависимости от контекста) присутствовать следующие расширения:

- **supported_versions**: При ведении переговоров TLS 1.3 основное поле версии сообщения ClientHello всегда содержит значение TLS 1.2 (x'0303), но в обязательном расширении `supported_versions` должно быть указано TLS 1.3 (x'0304).
- **signature_algorithms**: Строго говоря, в этом расширении определяется алгоритм (алгоритмы) подписи, который клиент может поддерживать в сообщении CertificateVerify (5), а в расширении **signature_algorithms_cert** — алгоритмы подписи, которые могут применяться в сообщении **Certificate (4)**. Однако, если расширение **signature_algorithms_cert** отсутствует, то расширение **signature_algorithms** применяется к обоим ролям.
- **supported_groups**: Поддерживаемые методы, используемые для [обмена ключами по алгоритму Диффи-Хеллмана](#) в порядке предпочтения. Возможные варианты: эллиптические кривые ((EC)DHE) или конечное поле (DHE).
- **key_share**: Одна (или несколько) структур, каждая из которых идентифицирует поддерживаемую группу (входящую в `supported_groups`) и разделяемое (общее) значение для указанной группы. Таким образом клиент уменьшает количество сообщений, требуемых для согласования, предлагая один (или более) общих ключей (один из которых будет использован сторонами соединения для вычисления операционных ключей сессии). Если клиент не предоставил общий ключ для той группы, которую впоследствии выбрал сервер, то сервер отправит сообщение **HelloRetryRequest**, по сути, говорящее клиенту о том, что ему нужно снова отправить **ClientHello** с обязательным указанием общего ключа для той группы, которая задана в расширении **key_share** сообщения **HelloRetryRequest**.

- **server_name:** индикация имени сервера (Server Name Indication, SNI) была опциональной в предыдущих версиях TLS, но в TLS 1.3 стала обязательной. Содержит список полностью квалифицированных DNS-имён хостов тех ресурсов, к которым осуществляется доступ, например, example.com и/или www.example.com

2. **ServerHello (2):** Сообщение ServerHello TLS 1.3 содержит определённые фиксированные поля, многие из которых присутствуют в нём для поддержания обратной совместимости. Поддерживаемая версия/вариант протокола всегда установлена в TLS 1.2 (но сообщение должно включать расширение **supported_versions** со значением, установленным в TLS 1.3). Другие поля: выбранный набор шифров, случайное значение (отметка текущего времени) в 32 октета (которое в дальнейшем используется в качестве материала для формирования ключа), идентификатор сессии (всегда 0 для обратной совместимости), метод сжатия (всегда 0= NULL для обратной совместимости). Кроме того, должны (в зависимости от контекста) присутствовать следующие расширения:

- **supported_versions:** При ведении переговоров TLS 1.3 основное поле версии сообщения ServerHello всегда содержит значение TLS 1.2 (x'0303), но в обязательном расширении **supported_versions** должно быть указано TLS 1.3 (x'0304).
- **supported_groups:** Сервер может послать данное расширение с единственным значением, которое просто указывает предпочтительный для него алгоритм обмена ключами Диффи-Хеллмана. Этот алгоритм клиент может использовать при последующих подключениях к данному серверу.
- **key_share:** Разделяемый (или общий) элемент для выбранного алгоритма, указанного в расширении **supported_groups**. Если в сообщении ClientHello (1) не было значения key_share для той группы, которую выбрал сервер, то сервер посылает сообщение **HelloRetryRequest** с расширением key_share, содержащим единственную группу, выбранную из списка, предоставленного клиентом в расширении **supported_groups** сообщения **ClientHello**.

3. **EncryptedExtensions (3):** Обязательное сообщение, позволяющее серверу отправлять клиенту дополнительные расширения (не связанные с шифрованием). Данное сообщение защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа **рукопожатия** (peer_handshake_traffic_secret).

4. **CertificateRequest (3A):** Если сервер настроен на поддержку аутентификации клиента (или взаимной аутентификации), данное сообщение запрашивает у клиента отправку соответствующего сертификата, который будет проверен сервером (для этого у сервера должен быть соответствующий сертификат удостоверяющего центра и/или промежуточные сертификаты). Данное сообщение защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа **рукопожатия** (peer_handshake_traffic_secret).
5. **Certificate (4):** В наиболее распространённом случае сервер отправляет свой собственный сертификат для аутентификации самого себя. Данное сообщение защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа **рукопожатия** (peer_handshake_traffic_secret).
6. **CertificateVerify (5):** Вероятно, наиболее интересное из сообщений, поскольку с его помощью сервер доказывает, что у него есть закрытый ключ, связанный с его сертификатом, а клиент имеет возможность проверить, что у сервера есть этот ключ. Сообщение (содержащее копию сообщений сессии и некоторые данные фиксированного контекста) подписывается цифровой подписью с использованием алгоритма, указанного в расширении **signature_algorithm** сообщения ClientHello (1). Затем всё сообщение целиком защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа **рукопожатия** (peer_handshake_traffic_secret).
7. **Finished (6):** Инициировано сервером. Копия всех сообщений сессии. Данное сообщение защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа **рукопожатия** (peer_handshake_traffic_secret).
8. **Finished (7):** Инициировано клиентом. Копия всех сообщений сессии до этого момента. Данное сообщение защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа **рукопожатия** (peer_handshake_traffic_secret).

9. **Обмен объёмными данными (8):** Начинается передача и прием объёмных данных. Каждое сообщение защищается с помощью того алгоритма шифрования объёмных данных и MAC, о которых стороны договорились в процессе криптографического обмена (ClientHello/ServerHello) с использованием ключа протокола **записи** (peer_application_traffic_secret_0).
10. **NewSessionTicket (8A):** В любой момент после того, как клиент отправил сообщение Finished (7), сервер может послать данное сообщение, содержащее значение, которое может быть использовано клиентом в качестве **pre_shared_key** (PSK, предварительного общего ключа) когда он [возобновляет \(или перезапускает\) данную сессию](#). Это сообщение является опциональным, серверы не обязаны отправлять его. Сервер может отправить более одного такого сообщения.

Примечания:

1. Отдельные сообщения обычно объединяются в блоки там, где такое объединение разумно. Так, например, сообщения 3, 3A (если есть), 4, 5 и 6, как правило, будут отправлены в виде одного блока.
2. В TLS 1.3 обязательным стало применение обмена по алгоритму Диффи-Хеллмана с использованием эфемерных ключей (либо на эллиптических кривых, либо в конечном поле). Это достигается путем повторного вычисления структуры key_share при каждом запуске (или перезапуске) сессии.
3. Ключ протокола **записи** может периодически обновляться. Для этого либо клиент, либо сервер посылает сообщение **KeyUpdate** в любой момент после отправки сообщения Finished (7). Обе стороны производят повторное вычисление ключа peer_application_traffic_secret_N (где N означает N-й вычисленный ключ) и сразу же начинают использовать этот новый ключ.
4. Любая из сторон может разорвать соединение с отправкой описательного (обычно) сообщения оповещения. Сообщения оповещения шифруются с использованием одного из ключей, в зависимости от этапа сессии, на котором происходит оповещение (peer_handshake_traffic_secret или peer_application_traffic_secret_N).

Возобновление (перезапуск) TLS 1.3 (рисунок 2B)

После того, как сессия была установлена с использованием [полного протокола рукопожатия TLS 1.3](#), она впоследствии может быть перезапущена (на жаргоне это называется возобновлением) с использованием метода, показанного на рисунке 2B, если и только если сервер отправлял одно или несколько необязательных сообщений **NewSessionTicket** (рисунок 2A, элемент 8A). В RFC не дается никаких

рекомендаций относительно того, когда следует использовать возобновление вместо полного рукопожатия. Хотя некоторые сообщения (в частности, **Certificate** и **CertificateVerify**) могут быть опущены, количество пересылок остается тем же, а количество обязательных расширений в **ClientHello** (1) увеличивается. Однако тот факт, что сервер может начинать отправку данных по протоколу записи (5) сразу после того, как пошлёт сообщение Finished (4), может дать некоторое преимущество в определенных обстоятельствах, например, после сбоя связи, когда сервер сохраняет информацию о состоянии сессии. **Примечание:** Существует дополнительная процедура (в RFC она называется 0-RTT), позволяющая осуществлять более быстрый перезапуск (возобновление) за счёт отказа от обеспечения *прямой секретности* (если какой-то из ключей был скомпрометирован, все соединения, возобновлённые с использованием процедуры 0-RTT, могут быть расшифрованы); в дальнейшем в этом руководстве она не обсуждается.



Рисунок 2В - Последовательность протокола возобновления сеанса TLS 1.3

Примечания:

1. Возобновление сессии зависит от того, согласны ли клиент и сервер использовать предварительный общий секрет (Pre Shared Secret), полученный из сообщения NewSessionTicket и отправленный обеими сторонами в расширении **pre_shared_key** (PSK) соответствующих сообщений Hello (1) и (2). Расширения **key_share** от обеих сторон обеспечивают прямую секретность, поскольку они важны для организации [Эфемерного обмена по алгоритму Диффи-Хеллмана](#) (DHE) как с использованием эллиптических кривых, так и конечных полей.

2. При возобновлении сессии сообщение ClientHello (1) должно включать расширение Server Name Indication (sni). Кроме того, последним расширением должно быть **pre_shared_secret**.
3. Сервер может начать посылать данные (5) сразу после отправки им сообщения **Finished** (4), поскольку предполагается, что клиент, раз уж он предоставил PSK, имеет всю необходимую информацию для расшифрования данных. Однако, если сообщение **Finished** (6) от клиента не будет получено, то сессия будет прервана сервером.
4. Если клиент в какой-то момент времени хочет перезапустить сессию снова, он должен использовать новые данные, предоставляемые в сообщении **newSessionTicket** (7A), отправленном в рамках уже перезапущенной сессии.

TLS 1.2, TLS 1.1/SSL — детальное описание (рисунок 2A)

В данном разделе приводится более подробное описание обмена сообщениями протоколов TLS версий 1.1 и 1.2 (смотрите рисунок 2A) для тех, кто любит покопаться во внутренностях (TLS 1.3 имеет значительные отличия и описывается [здесь](#)). Если Вам удобнее мириться с тем, что "в мире происходит много необычного", лучше [пропустите этот раздел](#), чтобы не рисковать рассудком.

Последовательность обмена сообщениями протоколов TLS 1.1 и 1.2/SSL



Примечания:

1. В фазе протокола рукопожатия проводятся переговоры и устанавливается соединение, а в фазе протокола записи происходит передача (инкапсуляция) зашифрованного потока данных, таких как HTTP, SMTP или IMAP.
2. На рисунке 2А чёрными стрелками показаны сообщения, отправляемые открытым текстом (незашифрованные); голубыми — сообщения, отправляемые с использованием открытого ключа, предоставленного сервером (с помощью шифра обмена ключами), для их обработки требуется, чтобы у сервера был доступ к соответствующему закрытому ключу; зелёными — сообщения, отправляемые с использованием того алгоритма шифрования объёмных данных (который будет использовать вычисленный общий секретный ключ) и того MAC, о которых стороны договорились в процессе переговоров.
3. На рисунке 2А сообщения для ясности показаны как отдельные объекты, на практике они обычно объединяются в блоки.
4. Версии TLS/SSL, предшествующие TLS 1.3, позволяют договориться об использовании какого-либо алгоритма сжатия данных (как составной части набора шифров). С учётом скорости современных сетей сжатие данных использовалось всё реже или вообще не использовалось, и обычно значение выбранного алгоритма сжатия в согласованном наборе шифров устанавливалось в NULL (не используется). И действительно, сжимать данные стали настолько редко, что в TLS 1.3 эту опцию вообще убрали, и все вздохнули с облегчением.

Последовательность обмена сообщениями

1. **ClientHello (1):** Сообщение ClientHello предлагает список поддерживаемых версий/вариантов протоколов, поддерживаемые наборы шифров в порядке предпочтения и список алгоритмов сжатия (обычно NULL). Клиент также посылает случайное значение размером в 32 байта (отметка текущего времени), которое в дальнейшем будет использоваться для обеспечения ключевого материала, и идентификатор сессии, который будет равен нулю, если не было предыдущих сессий, либо ненулевому значению, если клиент считает, что предыдущая сессия существует.
 - Каждый набор шифров состоит из алгоритма обмена ключами, алгоритма шифрования объёмных данных и алгоритма MAC (хэширования).
 - Набор шифров, используемый и клиентом и сервером, при первоначальном соединении таков:

```
TLS_NULL_WITH_NULL_NULL (0x00, 0x00)
# первый NULL – алгоритм обмена ключами
# следующий за ним WITH_NULL определяет алгоритм шифрования объёмных данных
# последний NULL определяет MAC
```

Это значение говорит о том, что шифрование выполняться не будет и потому все сообщения до Client Key Exchange (ClientKeyMessage) будут отправляться открытым текстом.

- Типичный набор шифров:

```
TLS_RSA_WITH_3DES_EDE_CBC_SHA (0x00, 0x0A)
# RSA — алгоритм обмена ключами
# WITH_3DES_EDE_CBC определяет алгоритм шифрования объёмных данных
# (Triple DES с циклической цепочкой блоков)
# SHA — это MAC (хэш)
```

Примечания:

1. Каждому набору шифров присвоен свой код. Значение этого кода (два октета) называется Сигнальным значением набора шифров (Signaling Cipher Suite Value, SCSV) — ещё одна фишка для Вашего гик-лексикона.
2. Корректные значения наборов шифров можно найти в приложениях C RFC по TLS ([RFC 2246](#) для TLS 1, [RFC 4346](#) для TLS 1.1 и [RFC 5246](#) для TLS 1.2). Значения для ECC (шифров на основе эллиптических кривых) приведены в [RFC 4492](#) и [RFC 7027](#). Скрупулёзнейший список шифров ведётся в [Реестре TLS IANA](#).
3. На [этой странице](#), касающейся настроек OpenLDAP, можно найти довольно детальное описание наборов шифров и методик манипуляции ими.
4. Слово EXPORT, встречающееся в описаниях некоторых наборов шифров, говорит о том, что эти шифры являются экспортируемыми, то есть некоторые шифры разрешено использовать только в определённых странах (смотрите документацию [Бюро промышленности и безопасности \(BIS\) Министерства торговли США](#) и [Вассенарские соглашения](#)), и это необходимо учитывать при настройке системы, которая будет использоваться на международном уровне.
5. Расширения, определённые в [RFC 3546](#) и в основном используемые в беспроводных сетях, могут быть применены в сообщении ClientHello на манер обеспечения обратной совместимости. [RFC 6066](#) значительно увеличивает количество расширений TLS, многие из которых могут использоваться в обычных (не беспроводных) сетях. Сервер волен молча проигнорировать те расширения, которые он не понимает.

6. В [RFC 6066](#) представлен тип расширения TLS Certificate Status (status_request), в котором, по существу, сказано: "Я (клиент) абсолютно не доверяю твоему (сервера) сертификату, но я буду абсолютно доверять твоему (сервера) ответу на мой запрос (Certificate Status) о валидности сертификата(!)". Ответ на запрос Certificate Status (обычно получаемый с помощью [OCSP](#)) посылается в сообщении **CertificateStatus** сразу после сообщения **Certificate** (смотрите ниже). Видимо, запросы Certificate Status (status-request) стали настолько популярны (среди беспроводных устройств?), что есть риск падения OCSP-серверов. В [RFC 6961](#) представлено расширение "certificate_request_v2", снижающее объёмы трафика от сервера TLS серверу OCSP путём разрешения первому из них кэшировать ответы OCSP, а также от сервера TLS клиенту TLS путём разрешения первому из них отправлять всю необходимую информацию, в том числе о промежуточных сертификатах, в одном сообщении **CertificateStatus**.
- В [RFC 6066](#) определено опциональное расширение Server Name Indication (SNI), позволяющее клиенту при установке первоначального соединения TLS/SSL посылать имя сервера, такое как www.example.com. Данная возможность (поддерживаемая большинством современных браузеров) позволяет web-серверу, обслуживающему несколько web-сайтов, (например, Apache с настроенной функцией VirtualHost) отправлять специфичный для сайта сертификат в своём сообщении **Certificate** (3). [Конфигурация Apache 2 для поддержки SNI](#).
 - В [RFC 7250](#) определено расширение **client_certificate_format**, которое может применяться для указания формата используемого сертификата. Это может быть нормальный формат X.509 или формат **RawPublicKey**, при котором в последующих сообщениях передачи сертификата протокола рукопожатия сертификат сокращается до одного атрибута [subjectPublicKeyInfo](#).

- Многие TLS/SSL-клиенты при возникновении сетевых ошибок будут пытаться произвести откат до более низкой версии протокола, что может привести к необоснованному ослаблению соединения. В [RFC 7507](#) определён новый шифр:

TLS_FALLBACK_SCSV {0x56, 0x00}

TLS/SSL-клиенты могут включать это сообщение в любую попытку переговоров о понижении версии протокола. Старые сервера проигнорируют такое сообщение, и будет нормальным способом устанавливаться соединение с более низкой версией протокола. Новые сервера, распознающие это сообщение, будут прерывать соединение с клиентом и выдавать предупреждение **inappropriate_fallback (86)**, если предлагаемая клиентом версия протокола ниже, чем та, что поддерживается сервером. Таким образом ограничиваются случаи, когда возможно установление соединения с необоснованным снижением версии протокола.

- В [RFC 7685](#) определено расширение, которое может быть использовано для дополнения (нулями) размера сообщения ClientHello в целях смягчения эффекта от работы ошибочных реализаций TLS (такого мы не ещё не пробовали).
- В [RFC 7633](#) определено новое [расширение сертификата X.509](#), которое включает в себя список расширений TLS, поддерживаемых этим сертификатом. Если сервер не предоставляет указанных расширений TLS, клиент может сделать предположение о потенциальной небезопасности сессии и прервать её. Очевидно, что такое решение клиент должен принять не позднее окончания фазы [Certificate](#) рукопожатия TLS.

2. **ServerHello (2)**: Сообщение ServerHello возвращает выбранные вариант/номер версии протокола, набор шифров и алгоритм сжатия. Сервер посылает случайное значение размером в 32 байта (отметка текущего времени), которое позднее будет использоваться для вычисления симметричных ключей. Если идентификатор сессии в сообщении ClientHello был равен нулю, сервер создаст и вернёт идентификатор сессии. Если в сообщении ClientHello был предложен идентификатор предыдущей сессии, известный данному серверу, то протокол рукопожатия будет проведён по упрощённой схеме. Если клиент предложил неизвестный серверу идентификатор сессии, сервер возвращает новый идентификатор сессии и протокол рукопожатия проводится по полной схеме.

В [RFC 7250](#) определено расширение **server_certificate_format**, которое может применяться для указания формата используемого сертификата. Это может быть нормальный формат X.509 или формат **RawPublicKey**, при котором в последующих сообщениях передачи сертификата протокола рукопожатия сертификат сокращается до одного атрибута [subjectPublicKeyInfo](#).

3. **Certificate (3)**: Сервер посылает свой [сертификат X.509](#), содержащий открытый ключ сервера, алгоритм которого должен совпадать с алгоритмом обмена ключами в выбранном наборе шифров. Протокол предлагает и другие методы доставки открытого ключа, — можно, например, просто указать на запись DNS KEY/TLSA RR, — но сертификат X.509 является стандартным общепринятым методом. Цель данного сообщения — получение клиентом из доверенного источника открытого ключа сервера, который затем может использоваться этим клиентом для отправки зашифрованного сообщения.

Примечания:

1. Хотя в этом сообщении обычно посылается только один сертификат, можно послать и так называемую связку сертификатов (certificate bundle) — более одного сертификата в PEM-файле. Например, связки сертификатов могут быть определены с помощью директивы Apache **SSLCertificateChainFile**, тогда как единичный сертификат должен быть определён директивой **SSLCertificateFile**. Обычно связки используются при наличии [кросс-сертификатов](#), появляющихся в результате некоторой формы реструктуризации сертификата, например, при корпоративном поглощении одной компании другой или изменении ключа/размера ключа удостоверяющего центра (CA).
2. Протокол DNSSEC DANE ([RFC 6698](#)) позволяет клиентам получать копию сертификата X.509 сервера с помощью запроса к системе DNS. Однако, сертификаты, полученные по системе DNS с помощью DANE, являются дополнительными по отношению к тем, которые получены в процессе нормального обмена сертификатами TLS/SSL, и служат скорее для того, чтобы патологические параноики могли провести дополнительную верификацию, а пользы в плане повышения безопасности приносят немного (если вообще приносят).
3. [RFC 7250](#) определяет упрощённый формат сертификата, в котором открытый ключ в чистом виде инкапсулируется в обёртку, состоящую из атрибута [SubjectPublicKeyInfo](#) (необходимого для описания свойств открытого ключа). Это скромный шаг в сторону более разумного способа обретения открытого ключа непосредственно от аутентифицированного источника, такого как защищённая DNS-запись DNSSEC.

4. **ServerDone (4)**: Это сообщение указывает на окончание серверной части данной последовательности диалога и побуждает клиента продолжить последовательность протокола. **Примечание:** В этом месте сервер может запросить клиентский сертификат для выполнения взаимной аутентификации. Последовательность обмена клиентским сертификатом была опущена из описания последовательности протокола, поскольку обычно она не используется и её включение усложнило бы данное описание.

Примечание: Если во время переговоров об установлении TLS/SSL сервер запросил сертификат клиента, то клиент должен отправить свой сертификат (в том же формате, который определен для сервера, с тем исключением, что [RFC 6066](#) позволяет любому клиенту посылать URL сертификата вместо полного сертификата) непосредственно за сообщением **ServerDone** и до сообщения **ClientKeyExchange**.

5. **ClientKeyExchange (5):** Клиент вычисляет так называемый ключ pre-master key, используя случайные числа (или отметки текущего времени) сервера и клиента. Он шифрует этот ключ с помощью открытого ключа сервера, полученного из предоставленного сертификата X.509. Только сервер может расшифровать данное сообщение своим закрытым ключом. Обе стороны независимо друг от друга вычисляют общий секретный ключ master key из ключа pre-master, используя определённый в стандарте алгоритм. Любые сеансовые ключи, которые могут потребоваться, будут производными от этого ключа master key.

Примечания:

1. Как известно, TLS (и DTLS) могут быть уязвимы к атакам типа "человек посередине" (Man-in-The-Middle, MTM). Для устранения этой уязвимости в [RFC 7627](#) был переопределён метод вычисления ключа master secret (изначально определённый в [RFC 5246](#)). В новом методе вместо случайных чисел сервера и клиента используется хэш полной сессии от сообщения **ClientHello** до сообщения **ClientKeyExchange**. Клиент демонстрирует, что он способен использовать новый (переопределённый) алгоритм вычисления master secret, путём отправки в **ClientHello** пустого расширения **extended_master_secret**. Если сервер поддерживает новый алгоритм, он также отправит пустое расширение **extended_master_secret** в сообщении **ServerHello**. Если клиент или сервер не поддерживают новый алгоритм (RFC 7627), они, конечно же, могут прервать сессию, либо продолжить её с использованием старого алгоритма (RFC 5246).
 2. При таком использовании открытого/закрытого ключей не обеспечивается прямая секретность. То есть, если закрытый ключ был скомпрометирован (украден, получен или взломан), то все ранее записанные сессии TLS могут быть расшифрованы. [Модифицированная процедура рукопожатия TLS 1.3](#) решает эту проблему.
6. **ChangeCipherSpec — клиент (6):** Это сообщение указывает, что весь последующий трафик, исходящий от данного клиента, будет зашифрован с помощью выбранного (в результате переговоров) алгоритма шифрования объёмных данных и будет содержать MAC, сформированный по выбранному алгоритму. Номинально это сообщение всегда будет шифроваться текущим шифром, который в стадии установления соединения будет NULL, и, следовательно, данное сообщение отображено на диаграмме как отправляемое в открытом виде. Несмотря на то, что это сообщение показано на диаграмме протокола как отправляемое отдельно, часто оно объединяется с сообщением Client Key Exchange.

7. **Finished — клиент (7):** Это сообщение содержит все сообщения, отправленные и полученные во время протокола рукопожатия, за исключением сообщения Finished. Оно шифруется с помощью алгоритма шифрования объемных данных и хэшируется с помощью алгоритма MAC, о которых договорились стороны. Если сервер может расшифровать и верифицировать это сообщение (содержащее все предыдущие сообщения), используя независимо вычисленный им сеансовый ключ, значит диалог был успешным. Если же нет, на этом месте сервер прерывает сессию и отправляет сообщение Alert с некоторой (возможно, неконкретной) информацией об ошибке.

Примечание: В [RFC 7918](#) указано, что при определённых условиях клиент может начать передачу данных сразу же после отправки данного сообщения для сокращения времени ожидания соединения. Если при обработке последующих сообщений от сервера произойдёт ошибка, то соединение будет разорвано, но данные скомпрометированы не будут.

8. **ChangeCipherSpec — сервер (8):** Это сообщение указывает, что весь последующий трафик, исходящий от данного сервера, будет зашифрован с помощью выбранного (в результате переговоров) алгоритма шифрования объемных данных и будет содержать MAC, сформированный по выбранному алгоритму. Номинально это сообщение всегда будет шифроваться текущим шифром, который в стадии установления соединения будет NULL, и, следовательно, данное сообщение отображено на диаграмме как отправляемое в открытом виде. Получение данного сообщения неявно говорит клиенту о том, что сервер получил и смог обработать сообщение Finished этого клиента.

9. **Finished — сервер (9):** Это сообщение содержит все сообщения, отправленные и полученные во время протокола рукопожатия, за исключением сообщения Finished. Оно шифруется с помощью алгоритма шифрования объемных данных и включает MAC, о которых договорились стороны. Если клиент может расшифровать это сообщение, используя независимо вычисленный им сеансовый ключ, значит диалог был успешным. Если же нет, клиент прерывает соединение и выдаёт сообщение Alert и подходящий (хотя и не всегда конкретный) код ошибки.

10. **Record Protocol (протокол записи):** Последующие сообщения между сервером и клиентом шифруются с помощью алгоритма шифрования объемных данных и включают MAC, о которых договорились стороны.

Примечания:

1. Случайные значения, посылаемые клиентом и сервером, и последующий секретный ключ pre-master включают в себя значение времени величиной два байта (для предотвращения атак повторения) и потому, как и во всех криптографических системах, и клиент и сервер должны использовать синхронизированный источник времени, такой как [NTP](#).
2. При прерывании соединения клиентом или сервером с выдачей сообщения Alert прилагаемый код ошибки может быть неточным (и, зачастую, бесполезным), чтобы не предоставлять другой стороне дополнительной информации, которую можно было бы использовать в последующих атаках.

Обзор сертификатов X.509 (SSL)

Оригинальный стандарт ITU-T X.509, в котором сертификаты получили своё несчастливое имя, — один из серии стандартов спецификации каталогов X.500. Использование сертификатов X.509 в Интернет стандартизировано IETF в [RFC 5280](#), определяющем формат сертификата X.509, а также в [RFC 4210](#), определяющем протокол управления сертификатами (Certificate Management Protocol, CMP), который используется для запросов на доступ и обработку сертификатов X.509 (есть также ряд дополнительных [протоколов](#), используемых для обработки и проверки сертификатов). Наконец, в [RFC 3739](#) определяется то, что называется **квалифицированным сертификатом (Qualified Certificate)**, относящимся к **Европейской директиве по электронным подписям (директива 1999/93/EC)**.

Для интересующихся: Стандарты каталога X.500 ITU-T определяли, помимо прочего, протокол DAP (Directory Access Protocol), который предназначался для поддержки злополучного почтового OSI-сервиса X.400. В IETF хотели получить сервис каталогов без всех этих OSI-наворотов и создали протокол LDAP (Lightweight Directory Access Protocol). Так что вся относящаяся к сертификатам X.509 техническая архитектура берёт своё начало из DAP/LDAP.

X.509 открывает совершенно новый и удивительный мир терминологии. В частности, для обозначения атрибутов в сертификате в нём используется термин "отличительное имя" (Distinguished Name, DN). DN определены IETF в [серии RFC по LDAP](#), конкретнее, в [RFC 4514 \(рус.\)](#). Также используются термины "абстрактная нотация синтаксиса 1" (Abstract Syntax Notation 1, ASN.1, по которой у нас есть отдельное [руководство по выживанию](#)) и "идентификатор объекта" (Object Identifier, OID), описанные в серии стандартов X.680 ITU. Наконец, при кодировании используются "особые (или уникальные) правила кодирования" (Distinguished Encoding Rules, DER), описанные в X.690 ITU.

Также существует большое количество связанных с управлением сертификатами стандартов, помеченных как PKCS#X (где X — число), например, PKCS#10

определяет формат запроса на подписание сертификата (Certificate Signing Request, CSR). Они относятся к стандартам, определенным [RSA Laboratories](#). Некоторые из этих стандартов были воспроизведены практически без изменений в качестве RFC, например, упомянутый выше PKCS#10 был опубликован как [RFC 2986](#) (обновлён в [RFC 5967](#)). В дополнение к стандартам от IETF, RSA и ITU-T, X.509 был стандартизирован в ряде стран, а также некоторыми промышленными организациями. Те, кто следит за этим вопросом, а также те, кто занимается реализацией, обращают внимание на то, что подобная множественность стандартов может привести к серьезным проблемам в реализации и интерпретации. Вот довольно подробный, большой и пугающий [набор заметок по реализации X.509 от Peter Gutmann](#).

Сертификат X.509 выполняет две различные функции:

1. Сертификат X.509 (в настоящее время X.509v3) служит контейнером для [открытого ключа](#), который может быть использован для проверки или валидации **конечного субъекта (end entity, EE)**, такого как веб-сайт или LDAP-сервер. Этот субъект определён в атрибуте [subject](#) сертификата, либо, всё чаще, в расширении [subjectAltName](#) (SAN). Субъект в атрибуте **subject** описывается в форме уникального или отличительного имени (Distinguished Name, DN), — [широко используется в LDAP](#), — состоящего из ряда относительных уникальных имён (Relative Distinguished Name, RDN), каждое из которых представляет собой содержащий данные элемент, называемый атрибутом (Attribute). В частности, атрибут CN (commonName), образующий RDN уникального имени, обычно содержит описание **конечного субъекта**, которому выдан сертификат. Примером CN может быть адрес веб-сайта, такой как CN=www.example.com. Полное DN в атрибуте **subject** или расширении **subjectAltName** может содержать один или более из следующих RDN: CN= (commonName, общепринятое имя, наименование конечного субъекта, например, website или www.example.com), C= (страна), ST= (штат или область в составе страны), L= (местоположение, номинально определяет адрес, но используется в разных целях, за исключением сертификатов EV, где его предназначение строго определено), OU= (organizationalUnitName, название подразделения компании или какая-либо иная подструктура), O= (organizationName, обычно название компании).

2. Сертификат X.509 подписан цифровой подписью доверенной удостоверяющей организации (обычно называемой удостоверяющим центром (Certificate Authority) или просто CA), уникальное имя (DN) которой указано в атрибуте [issuer](#) сертификата. Это делается, во-первых, для того, чтобы убедиться, что данный сертификат не был подделан, а во-вторых, для того, чтобы подтвердить (удостоверить), что данный открытый ключ для субъекта, указанного в атрибуте [subject](#) (или в расширении **subjectAltName (SAN)**) на самом деле является открытым ключом для данного **субъекта**. Этот процесс **доверия** мы опишем [далее](#). Подписывающая организация может быть удостоверяющим центром (Certification Authority, CA), регистрационным центром (Registration Authority, RA) или каким-то другим промежуточным центром (таким как подчинённый удостоверяющий центр (subordinate CA)), кроме того сертификат может быть [самоподписанным](#). **Примечание:** Закрытый ключ, ассоциированный с открытым ключём в сертификате X.509 пользователя, всегда хранится у пользователя и никогда не разглашается подписывающей организации.

Примечание: Несмотря на то, что сущность, описанная в атрибутах **subject**, **subjectAltName**, **issuer** (или даже **issuerAltName**), чаще всего представляет собой DN, тип этих атрибутов определяется как **GeneralName**, в котором может быть представлен ряд форматов, включая DN (**directoryName**), адрес электронной почты (**rfc822Address**), имя домена DNS (**dNSName**), а также всеохватывающий тип **otherName**, в котором может дополнительно определяться требуемый формат. До сих пор всё казалось таким простым.

Отдельная структура X.509, называемая [списком аннулированных \(или отозванных\) сертификатов \(Certificate Revocation List, CRL, в настоящий момент CRLv2\)](#) предоставляет информацию о сертификатах, которые по тем или иным причинам были аннулированы или признаны недействительными. По сути, CRL представляют собой старомодный "пакетный" способ работы с аннулированными сертификатами. В них содержатся большие (порой даже очень большие) списки всех сертификатов, которые были отозваны. Если проверяемого (по серийному номеру) сертификата нет в списке CRL, предполагается, что он ещё действителен. У списков CRL есть множество проблем: они могут обновляться только периодически и только УЦ (издателем сертификатов), из-за большого размера CRL браузеры скачивают их (если вообще это делают) неохотно и нерегулярно. Короче говоря, это не очень удобный и эффективный способ. Для проверки текущего статуса конкретного сертификата (опять же, определяемого по серийному номеру) всё чаще и чаще используется онлайн-способ ([OCSP](#)), а спецификация SSL-сертификатов EV, вообще требует обязательного использования OCSP.

Большая часть информации в этом разделе сфокусирована на использовании X.509 для валидации коммуникаций сервера. X.509 может также использоваться и для других целей, таких как идентификация пользователей (включая цифровые подписи)

и S/MIME, которые мы обязательно рассмотрим, когда (если) голова болеть перестанет.

Типы сертификатов X.509 и терминология

При обсуждении вопросов, связанных с сертификатами X.509 (SSL), используется несметное количество терминов. Иногда они применяются последовательно, но в основном — нет, что только добавляет неразберихи. Даже в RFC, посвящённым сертификатам, нет строго определения терминологии, пожалуй, лучше всего с этим обстоят дела в [RFC 4210](#). Как правило, удостоверяющие центры (УЦ) предлагают несколько типов сертификатов. За исключением [сертификатов EV](#) и **квалифицированных сертификатов**, имеющих точное назначение и определение, все эти типы сертификатов, по большому счёту, — маркетинговая концепция, цель которой — предоставить выбор по цене/функциональности. Поэтому мы ограничимся лишь поверхностным обзором подобных типов сертификатов, в лучшем случае предоставляя немного технической терминологии. Наконец, не все УЦ одинаковы. Хотя большинство УЦ являются нарочито профессиональными организациями, которые регулярно проверяются или сертифицируются теми или иными национальными организациями, это не всегда так (на сайте УЦ ищите ссылки по аттестации, сертификации и аудиту и смотрите, что там). При покупке сертификата необходимо проявлять бдительность!

Далее предпринята попытка определить наиболее часто используемые термины по удостоверяющим центрам и типам сертификатов в свете того, что было сказано в предыдущем параграфе. Предлагаемые пояснения взяты из технической документации (в основном из RFC, особенно RFC 4210), а также с веб-сайтов удостоверяющих центров.

Удостоверяющий центр (УЦ, он же корневой УЦ, англ. Certificate Authority (CA) a.k.a. root CA): общий термин "удостоверяющий центр" определяется как субъект или организация, подписывающая сертификаты. Корневой УЦ — это УЦ генерирующий корневые сертификаты, имеющие следующие характеристики: атрибуты **issuer** и **subject** совпадают; в расширении **basicConstraints** атрибут **ca** установлен в TRUE; расширение **KeyUsage** установлено в **keyCertSign** (опционально). Как правило, в цепочке сертификатов сертификат корневого УЦ является сертификатом самого верхнего уровня, однако RFC 4210 определяет, что корневым УЦ может быть любой издатель (**issuer**), сертификат которого (с расширением **basicConstraints**, в котором **ca** установлен в TRUE) получен конечным субъектом (например, браузером) в результате доверенного процесса получения. Поскольку окончательное решение о выдаче любого сертификата остаётся за корневым УЦ, термины и условия любого промежуточного сертификата могут быть изменены данным субъектом.

Примечание: Мы определили Удостоверяющий центр (УЦ) как эквивалент корневому УЦ. На что нам было справедливо указано, что это не совсем так. Существуют промежуточные и подчинённые удостоверяющие центры (определены ниже), которые не выдают корневые сертификаты. Мы не стали менять своего определения из-за общепринятого использования данного термина, однако считаем своим долгом предупредить читателей, что термин Удостоверяющий центр (УЦ) является общим, и может быть отнесён, к примеру, и к корневому, и к подчинённому УЦ.

Регистрационный центр (РЦ, он же регистрационный УЦ, англ. Registration Authority (RA) a.k.a. Registration CA): Регистрационный центр (RA) может потребоваться в определенных условиях для обработки специфических характеристик сертификата, например, РЦ может быть делегирован Национальным удостоверяющим центром для специализации на персональных сертификатах, а другой РЦ может специализироваться на сертификатах серверов. РЦ, если таковые вообще имеются, являются, по сути, неким средством для административного удобства. РЦ могут подписывать сертификаты (как подчинённые УЦ), но обычно, выполнив соответствующие проверки конечного субъекта, передают запрос на подписание корневому УЦ.

Подчинённый центр (подчинённый УЦ, англ. Subordinate Authority a.k.a. Subordinate CA): Общий термин. Любой субъект, подписывающий сертификаты, но не являющийся корневым УЦ. Некоторые подчинённые УЦ - особенно те, которые работают под полным контролем владельца корневого УЦ - могут быть помечены как УЦ (будет присутствовать расширение **BasicConstraints** и **сА** будет установлено в **True**). То есть, если РЦ подписывает сертификаты, то он делает это как подчинённый УЦ, и если он работает под контролем корневого УЦ, то он также может быть помечен, как УЦ.

Промежуточный центр (промежуточный УЦ, англ. Intermediate Authority a.k.a. Intermediate CA): Неточный термин, иногда используется для определения субъекта, создающего промежуточные сертификаты, и, таким образом, охватывает РЦ и подчинённые УЦ.

Кросс-сертификаты (они же сертификаты сцепления или мостовые сертификаты, англ. Cross certificates a.k.a. Chain or Bridge certificate): Кросс-сертификат представляет собой сертификат, в котором субъекты в атрибутах **subject** и **issuer** не совпадают, но оба являются удостоверяющими центрами (присутствует расширение **BasicConstraints** и атрибут **сА** установлен в **True**). Как правило, такие сертификаты используются, когда УЦ изменил некоторые элементы своей политики издания сертификатов (новая дата истечения срока действия ключа или новый ключ), либо когда один УЦ был поглощён другим, и сертификаты, изданные поглощённым УЦ, привязываются к новому владельцу, чтобы можно было отказаться от использования ранее выданных корневых сертификатов. При использовании в

данном контексте термин сертификат сцепления указывает на то, что была создана новая привязка, и иногда называется мостовым сертификатом (он привязывается к новому УЦ или выполняет функции моста к нему). Кросс-сертификаты могут быть установлены на сервере (как часть связки сертификатов — смотрите примечание в разделе [Протокол TLS, сообщение Certificate](#)), но при использовании для обеспечения обратной совместимости, например, при обработке EV-сертификата несовместимым с EV клиентом, кросс-сертификат устанавливается на клиенте. За исключением того, что атрибут **cA** установлен в **True** (что, откровенно говоря, мало что меняет) кросс-сертификат представляет собой обыкновенный промежуточный сертификат.

Промежуточные сертификаты (они же сертификаты сцепления, англ.

Intermediate certificates a.k.a. Chain certificates): Неточный термин, применяемый к любому сертификату, не подписанному корневым УЦ. Промежуточные сертификаты формируют [цепочку](#), в которой на пути от сертификата конечного субъекта до корневого сертификата может быть сколько угодно промежуточных сертификатов. Промежуточные сертификаты могут быть выпущены подчинёнными УЦ, РЦ или даже напрямую корневым УЦ (хотя технически их следует называть кросс-сертификатами) для различных целей: организации перехода, поглощения или даже просто для дифференциации брендов. В данном контексте термин **сцепление** бессмысленен (хотя звучит умно и помпезно), он просто указывает на то, что сертификат является частью какой-то цепочки.

Связка сертификатов (англ. Certificate Bundle): Общий термин, указывающий на то, что в один файл (обычно в формате [PEM](#)) объединены несколько сертификатов X.509. Связки сертификатов могут передаваться во время протокола рукопожатия TLS/SSL. Обычно связки сертификатов используются для административных целей во время некоторых изменений состояния УЦ, например, поглощения, изменения политики, ключа, даты истечения срока действия ключа и т.п.

Квалифицированные сертификаты (англ. Qualified certificates): Определённый в [RFC 3739](#) термин "квалифицированные сертификаты" относится к персональным сертификатам (а не к сертификатам серверов или сертификатам конечного субъекта) и ссылается на Директиву Европейского союза об электронной подписи (1999/93/EC), сфокусированную на единообразном определении индивидуума в целях электронной подписи, авторизации или аутентификации. В частности, данное RFC позволяет содержать в атрибуте **subject** в порядке очередности атрибуты **commonName (CN=)**, **givenName (GN=)** или **pseudonym=**, также может присутствовать атрибут **subjectDirectoryAttributes**, содержащий какие-либо из атрибутов **dateOfBirth=**, **placeOfBirth=**, **gender=**, **countryOfCitizenship=** и **countryOfResidence=**. Наконец, в этом RFC определены два новых расширения **biometricInfo** и **Qualified Certificate statements (qcStatements)**. Квалифицированный сертификат определяется по наличию расширения **qcStatements** со значением **qcStatement-2**. Большинство

правительств конкретных стран добавили для включения в такие сертификаты ряд дополнительных атрибутов. В некоторых случаях на то были реальные причины, в других — просто желание продемонстрировать свои амбиции и сделать невыносимой жизнь тех, кто занимается реализацией стандартов сертификатов.

Неквалифицированные сертификаты (англ. non-Qualified certificates): Вообще, так называют персональные сертификаты, не удовлетворяющие требованиям стандарта квалифицированных сертификатов. Издатели **квалифицированных сертификатов** также употребляют этот термин по отношению к другим сертификатам с намёком на то, что эти сертификаты более низкого качества.

Сертификат конечного субъекта, он же листовой сертификат (англ. End-Entity Certificate a.k.a Leaf Certificate): Тут всё непросто. Термин "конечный субъект" (в английском варианте могут использоваться как end-entity, так и end entity) изначально определяется в X.509, а затем в [RFC 4949](#) и [RFC 5280](#). Во всех случаях смысл в том, что сертификат конечного субъекта — это сертификат, в котором для защиты конечного субъекта, описанного в атрибуте CN= атрибутов [subject](#) или [subjectAltName](#), используется закрытый ключ (соответствующий открытому ключу, указанному в сертификате конечного субъекта). С другой стороны, иногда этот термин используется для указания на то, что закрытый ключ (соответствующий открытому ключу, указанному в сертификате конечного субъекта) не используется для подписи сертификатов, то есть, сертификат конечного субъекта не является промежуточным сертификатом, как правило, не является корневым (CA) сертификатом, и, следовательно, не используется в каком-либо процессе проверки подписи. Термин "листовой сертификат" используется для указания на то, что сертификат конечного субъекта, как правило, является последним сертификатом в цепочке. Сами решайте, помогают подобные термины понимать происходящее, или только мешают этому.

Мультихостовые сертификаты (англ. Multi-host certificates): Сертификат сервера обычно в атрибуте **subject** содержит атрибут CN=hostname, например, CN=www.example.com. Имя хоста разрешается системой DNS, которая может выдавать для этого хоста несколько IP-адресов (если в DNS имеется несколько [записей A или AAAA](#)). В этом случае серверный сертификат X.509 (SSL) с одним и тем же именем хоста может быть реплицирован на все подобные хосты (очевидно, соответствующий закрытый ключ также должен быть реплицирован на каждый хост, что может представлять собой проблему при использовании аппаратных криптографических устройств, в этом случае некоторые УЦ с удовольствием продадут Вам дополнительные сертификаты (которые будут называться мультихостовыми) чтобы обойти эту проблему). Если же присутствует несколько имён хостов, таких как www.example.com, example.com или www1.example.com, в таком случае требуется специальный подход и специальные типы сертификатов,

описанные ниже как **мультидоменные сертификаты** и **сертификаты с возможностью подстановки**.

Мультидоменные сертификаты, они же сертификаты SAN или UCC (англ. Multi-domain certificates a.k.a SAN or UCC certificates): Некоторые УЦ продают мультидоменные сертификаты, охватывающие несколько доменных имён, например, таких как **www.example.com**, **example.com** или даже **www.example.net**. Это достигается путём использования нескольких записей в атрибуте **subjectAltName** и описывается [далее](#). С технической точки зрения на этот процесс практически нет ограничений, например, в одном сертификате X.509 (SSL) могут поддерживаться **www.example.com** и **www.example.net**, но у большинства УЦ предусмотрены на это какие-либо коммерческие ограничения, которые, впрочем, обычно можно преодолеть путём дополнительных капиталовложений. Иногда для этих целей можно использовать описанные ниже **сертификаты с возможностью подстановки**, но имена хостов в таких сертификатах ограничены одним доменом.

Сертификаты с возможностью подстановки (англ. Wildcard certificates):

Некоторые УЦ продают сертификаты с возможностью подстановки, в которых атрибут **subject** содержит **CN=*.example.com** (* — это шаблон подстановки). Такие сертификаты поддерживают любое имя хоста в составе домена, то есть ***.example.com** будет поддерживать **www.example.com** и **mail.example.com**, но не **example.com** и, очевидно, не **example.net** (для этих целей могут подойти описанные выше [мультидоменные сертификаты](#)).

Сертификаты альтернативного имени субъекта (англ. Subject Alternative Name (SAN) certificates): Хотя технически этот термин более точен (в нём

подразумевается, что имя субъекта сертификата (обычно не одно) содержится в атрибуте **subjectAltName**, а не в атрибуте **subject**), за ним скрывается лишь ещё одно причудливое имя для [мультидоменных сертификатов](#). С точки зрения поставщиков, у сертификатов SAN есть одно большое преимущество: название звучит дороже.

Сертификат унифицированных коммуникаций (англ. Unified Communication Certificate (UCC)): Абсолютно бессмысленный термин, по сути, ещё одно причудливое имя для [мультидоменного сертификата](#).

Сертификаты EV (сертификаты расширенной валидации они же расширенные сертификаты, англ. EV (Extended Validation) Certificates a.k.a. Extended Certificates): Сертификаты EV отличаются наличием расширения **CertificatePolicies**, содержащего зарегистрированные OID в атрибуте **policyIdentifier**. Подробнее сертификаты EV описаны [ниже](#).

Сертификаты DV (сертификаты валидации домена, англ. Domain Validation (DV) Certificates): Некоторые УЦ выпускают то, что называется сертификатами валидации

домена (DV). Этот термин не используется повсеместно. Подразумевается, что удостоверяющий центр подтверждает только тот факт, что лицо или организация, запросившая данный сертификат, является владельцем доменного имени. То есть значение CN= в атрибутах **subject** или **subjectAltName**, например, www.example.com, может рассматриваться как верное, а вот информация об организации (C=, ST=, L=, OU= или O=) не должна рассматриваться как верная, и эти значения должны быть либо пустыми, либо содержать соответствующий текст, например, "not valid" ("неверно").

Сертификаты OV (сертификаты валидации организации, англ. Organizational Validation (OV) Certificates): Некоторые УЦ выпускают то, что называется сертификатами валидации организации (OV). Этот термин не используется повсеместно. Подразумевается, что удостоверяющий центр подтверждает тот факт, что лицо или организация, запросившая данный сертификат, является владельцем доменного имени, а также подтверждает предоставленные в сертификате сведения об организации. То есть значения CN=, C=, ST=, L=, OU= или O= в атрибутах **subject** или **subjectAltName** могут рассматриваться как верные. Хотя на первый взгляд это выглядит весьма основательно, всё же такие сертификаты не дотягивают до уровня [сертификатов EV](#), требующих дополнительных условий квалификации.

Внутридоменные сертификаты (англ. Domain-Only Certificates): Общий термин, обычно несёт негативную окраску, применяется к сертификатам, в которых процесс верификации конечного пользователя, с точки зрения тех, кто применяет данный термин, варьируется от поверхностного до вообще никакого.

Сертификаты защиты контента при цифровой передаче (Digital Transmission Content Protection, DTCP): Такие сертификаты выпускаются и управляются Администратором лицензирования цифровой передачи (Digital Transmission Licencing Administrator, DTLA — www.dtcp.com) и обычно используются Smart-телевизорами, медиа-плеерами и прочими подобными устройствами при проигрывании на них лицензионных материалов. Сертификаты DTCP не используют формат X.509, но они могут быть задействованы в протоколе рукопожатия TLS ([RFC 7562](#)). Далее в этом документе они не описываются.

Цепочки сертификатов X.509

Сертификаты X.509 могут формировать **цепочки**, то есть они могут быть подписаны одним или более промежуточными удостоверяющими центрами в иерархической манере, либо сертификат может быть просто подписан корневым УЦ напрямую. Концепция регистрационных центров (РЦ), выступающих в роли промежуточных подписывающих удостоверяющих центров, представлена в упомянутых выше RFC. Понятие "регистрационный центр (РЦ)" (иногда в стандартах EV именуемый "подчинённым УЦ"), вводится для описания организации, у которой на самом деле

был приобретён сертификат X.509, например, лицензированного агента, сертификат которого подписан (возможно, также в результате иерархической цепочки подписания) удостоверяющим центром, представляющим собой УЦ самого верхнего уровня или корневой УЦ. По структуре чем-то похоже на операторов реестра DNS и регистраторов, если организация DNS Вам знакома. В [RFC 4158](#) есть полезная, но крайне запутанная и неудобоваримая информация о том, как можно надёжно построить цепочку сертификатов с помощью пар атрибутов **subject** и **issuer**, дополняя их, кроме всего прочего, атрибутами **SubjectKeyIdentifier** и **AuthorityKeyIdentifier**.

Сертификат самого верхнего уровня в иерархии подписания называется корневым сертификатом, сертификатом УЦ или даже иногда корневым сертификатом УЦ. Корневые сертификаты получают каким-либо доверенным способом (в случае браузеров они распространяются с программным обеспечением браузера и периодически обновляются) и при валидации цепочки сертификатов обычно называются якорями доверия. При получении от сервера в процессе рукопожатия TLS/SSL сертификата (или связки сертификатов) конечного субъекта, программа, получившая сертификат, должна проверить его и весь путь до корневого сертификата (сертификата УЦ), включая, при необходимости, все промежуточные сертификаты (которые, опять же, обычно распространяются с программным обеспечением браузера). Корневой сертификат (сертификат УЦ) распознаётся по наличию одинаковых значений в атрибутах [issuer](#) и [subject](#), по тому, что атрибут [KeyUsage](#) установлен в **keyCertSign** и/или в атрибуте [BasicConstraints](#) атрибут **cA** установлен в TRUE. Процесс построения цепочки сертификатов описан в [RFC 4158](#), валидация цепочки сертификатов описана в [RFC 5280](#). Различные варианты подписания сертификатов показаны на рисунке 3:

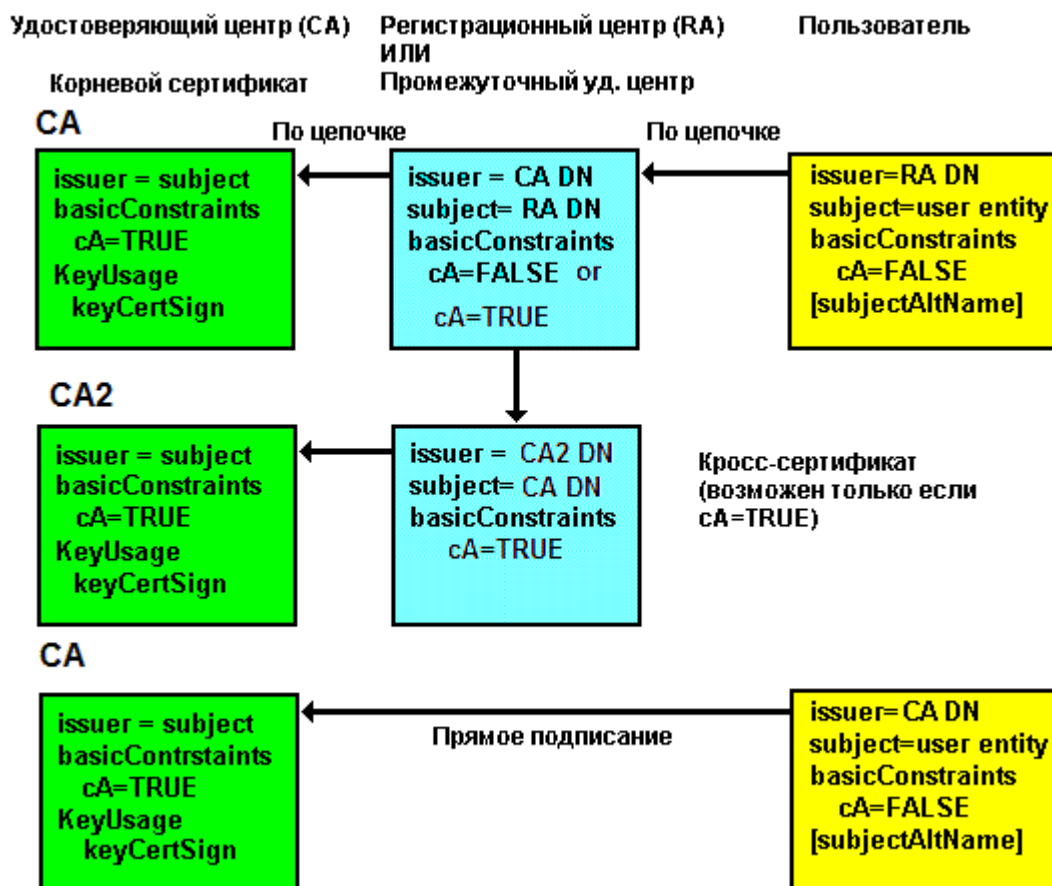


Рисунок 3 — Цепочки сертификатов X.509

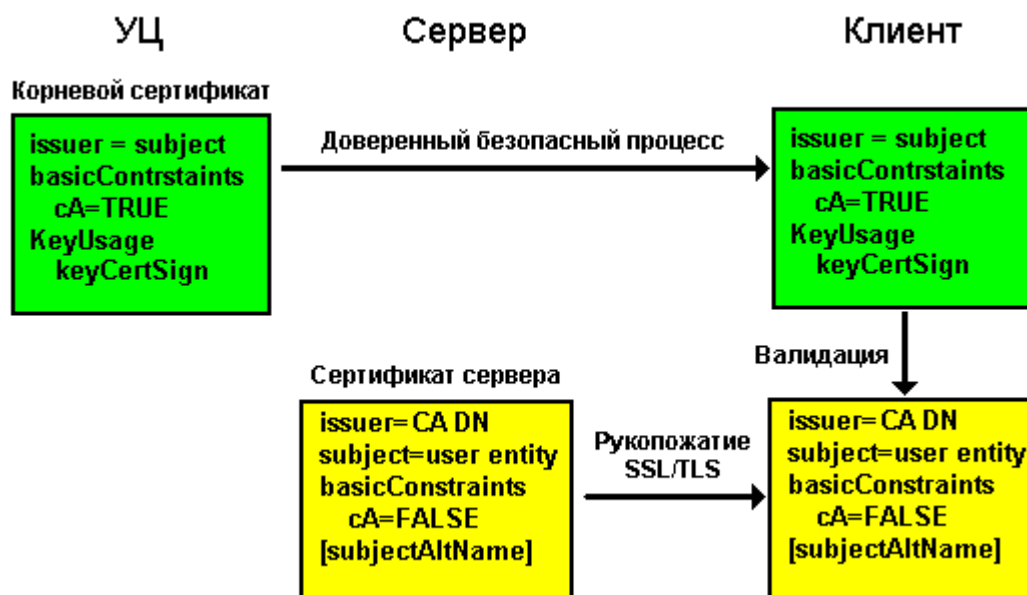
Использование сертификатов X.509

Издатель сертификата ([issuer](#)) идентифицируется с использованием формата Distinguished Name (DN, уникального имени), который в оригинале разрабатывался для представления расположения объекта в DIT (информационном дереве каталога) DAP или LDAP. DN не следует путать с сетевым адресом или URL/URI. Обычно DN имеет формат **CN=Type of Certificate, OU=Certificate Division, O=Certificate Company name, C=Country (CN=, OU=, O=, C=)**, но может иметь и более простой формат **OU=, O=, C=**, или даже **CN=, O=, C=**, наконец (чтобы немного запутать ситуацию) он может иметь формат **CN=, OU=, DC=, DC=**. В общем, формат может быть разным — вариантов много. DN состоит из нескольких разделённых запятыми RDN (Relative Distinguished Names, относительных уникальных имён), то есть CN= или C= представляют собой RDN в составе DN. Сертификат X.509 не содержит URI для получения по сети каких либо сертификатов для образования цепочки, но он может содержать (в других атрибутах) URI для получения [CRL](#). Использующие сертификаты приложения, — к примеру, браузеры или почтовые клиенты, — должны заранее получить корневой сертификат, а если существует цепочка сертификатов, то и все промежуточные сертификаты, каким-либо способом, по сети или нет. Корневые сертификаты наиболее известных УЦ распространяются с браузерами (и доступны для соответствующих им почтовых клиентов), смотрите раздел ["Работа с сертификатами в основных браузерах"](#).

Примечание: Распространяемые с программным обеспечением большинства браузеров (и почтовых клиентов) корневые сертификаты добавляются в соответствии с критериями, определяемыми поставщиком браузера, и варьирующимися от "кто больше заплатит" до тотальной проверки легитимности УЦ и соответствия их другим требованиям.

При обращении приложения к сервису с поддержкой TLS/SSL, в процессе [диалога рукопожатия TLS/SSL](#) оно получает сертификат (или связку сертификатов) сервера. После этого приложение (например, браузер) выделит атрибут CN из DN в атрибуте [subject](#) (и/или в атрибуте [subjectAltName](#), смотрите [RFC 6125](#)) для проверки субъекта (скажем, адреса веб-сервера, такого как `www.example.com`). Затем наше приложение будет использовать DN в атрибуте [issuer](#) сертификата X.509 сервера для нахождения соответствующего корневого сертификата в своём локальном хранилище (и, если такового не обнаружено, генерации исключения — обычно в виде непонятного, пугающего пользователя диалога). Если найден валидный корневой сертификат, то тем самым подлинность предоставленного сервером сертификата считается установленной. В итоге (если всё прошло успешно) открытый ключ, содержащийся в сертификате X.509, считается безопасным (доверенным) и пригодным для проведения коммуникаций с указанным в атрибуте `subject` субъектом.

Серверам обычно требуется только свой собственный сертификат (сертификаты) и не требуется корневых сертификатов — они просто посылают свои сертификаты клиенту, а валидации сертификатов не производят. Однако TLS/SSL позволяют проводить взаимную валидацию, в этом случае и сервер и клиент отправляют свои сертификаты. Если приложение требует, чтобы клиент предоставлял свой сертификат, то сервер должен будет произвести валидацию клиентского сертификата, для чего у него должны иметься все необходимые корневые и промежуточные сертификаты, полученные любым путём, например, по почте или с веб-сайтов удостоверяющих центров.



[Замечания по атрибутам сертификата subject и subjectAltName](#)

Сертификаты X.509 в организациях, предоставляющих услуги веб-хостинга

Владельцы веб-сайтов подвергаются значительному давлению по поводу внедрения шифрования на основе сертификатов X.509 (SSL). Источники этого давления обычно имеют благие намерения (но это не говорит о том, что они правы).

Когда владелец веб-сайта одновременно является и его оператором, такое внедрение не создаёт больших проблем. Владелец/оператор сайта просто должен определиться, является ли внедрение TLS экономически эффективным. С другой стороны, многие владельцы сайтов делегируют функции оператора сайта специалистам веб-хостинговых компаний (так называемые многопользовательские сайты). И тут может возникнуть более серьёзная проблема.

Так в чём же дело?

Проблема связана с выдачей сертификатов X.509, а также с владением закрытыми ключами, ассоциированными с этим сертификатом X.509. В частности, имеет место одна единственная (в большинстве случаев) транзакция (пункт 5 в [последовательности обмена сообщениями протокола TLS](#)), в которой требуется, чтобы у сервера был доступ к закрытому ключу соответствующего сертификата (напомним, что в асимметричной криптографии (с открытым и закрытым ключами) клиент шифрует сообщение с использованием открытого ключа, и только владелец закрытого ключа, — в данном случае сервер, — может расшифровать это сообщение).

Теперь представьте, что собственник example.com делегировал обслуживание веб-сайта с этим доменным именем веб-хостинговой организации, имеющей доменное имя example.net. Если браузер пользователя соединяется с TLS-сервисом на www.example.com, то он ожидает увидеть сертификат с именем www.example.com (то есть с именем того сайта, к которому он подключается). Если же он получит сертификат с именем www.example.net (доменное имя хостинговой организации), то будет немного расстроен (на самом деле он либо начнёт злорадно выдавать пользователю неприятные сообщения, либо подкрасит адресную строку угрожающим красным цветом).

Чтобы проблема не была такой острой, в [RFC 6066](#) была введена SNI (Server Name Indication, индикация имени сервера), позволяющая клиенту (браузеру) явно указать в сообщении **ServerHello**, что он подключается к www.example.com, давая

возможность нашему супер-умному серверу предоставлять ожидаемый сертификат (www.example.com). Победа! Браузер снова счастлив, ура!

Не так быстро.

Ни один нормальный удостоверяющий центр (CA) не выдаст сертификат для example.com какому-то там example.net. Только владелец домена (который должен тем или иным способом доказать своё право собственности) может получить сертификат на свой домен. Однако, как мы помним из пункта 5 [последовательности обмена сообщениями протокола TLS](#), серверный хост должен иметь у себя не только сертификат, но и ассоциированный с ним закрытый ключ. Ой! Юристы уже радостно потирают руки! Ой-ой-ой!

В [RFC 7711](#) предлагается решение, при котором пользователь (example.com) может (используя ресурсные записи [DNS SRV](#)) явно делегировать предоставление SSL-сертификата третьей стороне (в нашем случае example.net) для проверки подлинности своего сайта, путём указания на расположенную где-то на веб-хосте специально сформированную запись в формате JSON. Пользователю не нужно покупать безумно дорогой SSL-сертификат и, следовательно, не нужно предоставлять свой закрытый ключ хостинг-провайдеру. В теории, перед посещением веб-сайта пользователя наш браузер будет выполнять запрос DNS SRV, а затем, используя полученный URL, прочтёт запись делегирования (предположим, по логике нашего примера, в этой записи указано, что полномочия делегированы example.net). При получении этой информации, браузер с радостью примет сертификат от example.net при доступе к example.com. Наш браузер доволен, никаких неприятных сообщений и угрожающих цветов. Все счастливы, кроме, пожалуй, юристов.

Примечание: В этом же RFC опционально пользователю разрешается указать, что он намеренно предоставил своему хостинг-провайдеру свой сертификат X.509 (и, неявно, свой закрытый ключ).

У этой проблемы существует и другое возможное решение с использованием DNSSEC и DANE, когда-нибудь мы его тоже обязательно задокументируем. Только не надейтесь, что это будет скоро.

[Дополнительные замечания по атрибутам сертификата subject и subjectAltName](#)

Протоколы, связанные с сертификатами (CMP, CMC/CMS, CRMF, SCVP, OCSP, HTTP)

Сертификат X.509 представляет собой структуру данных. Различные протоколы позволяют производить манипуляции с сертификатами через коммуникационную сеть. Такие **операции с сертификатами X.509**, как отправка запроса на подписание, получение подписанного сертификата и другие, определены, в частности, в

протоколе управления сертификатами (Certificate Management Protocol, CMP, [RFC 4210](#)) и в PKCS #10 ([RFC 2986](#)), и описаны далее в этом руководстве.

Беглый обзор: Для манипуляций с сертификатами и списками отзыва сертификатов (Certificate Revocation Lists, CRL) определен ряд протоколов. Протокол управления сертификатами (Certificate Management Protocol, CMP, [RFC 4210](#), обновлён в [RFC 6712](#)) предоставляет методы для манипуляции сертификатами (формат сообщений сертификатов (Certificate Request Message Format, CRMF) определён в [RFC 4211](#)). В RFC 4210 определено несколько коммуникационных методов (таких как HTTPS), которые могут быть использованы для транспортировки запросов сертификата по сети, но не дано явного определения транспортного протокола для обработки сертификатов (смотрите CMC/CMS ниже).

Протокол CMC/CMS (Certificate Management over CMS, управление сертификатами поверх CMS), определён в [RFC 5272](#), [RFC 5273](#), [RFC 5274](#) и обновлён в [RFC 6402](#), позволяет осуществлять безопасную транспортировку запросов сертификата от запрашивающей стороны к УЦ (включая любые промежуточные ПЦ) и обратно. Формат сообщения может быть либо PKCS #10 ([RFC 2896](#)), либо CRMF ([RFC 4211](#)). Этот протокол следует называть CMC, но иногда его называют CMS. Технически, CMS — это Cryptographic Message Syntax (синтаксис криптографического сообщения), в котором описывается только формат конверта запроса и ответа (для PKCS #10 или CRMF). Данный протокол определяет ряд операций, которые могут быть выполнены над сертификатами, в том числе обновление якорей доверия (корневых сертификатов), поддерживаемых у запрашивающей сертификат стороны. Это большой и насыщенный (сложный) протокол.

Протокол онлайн-получения статуса сертификата (Online Certificate Status Protocol, OCSP, [RFC 6960](#)) — это протокол для онлайн-проверки подлинности сертификатов. [RFC 6066](#) расширяет стандарт TLS чтобы клиент мог запрашивать OCSP-статус сертификата во время фазы протокола рукопожатия, а [RFC 6961](#) определяет упрощённый формат 'certificate_request_v2', позволяющий снизить объёмы трафика сервера OCSP.

Протокол серверной валидации сертификатов (Server-Based Certificate Validation Protocol, SCVP, [RFC 5055](#)) позволяет делегировать недоверенному серверу некоторые клиентские функции, а именно построение и валидацию пути сертификации. Если же сервер SCVP является ещё и доверенным, то он может предоставлять дополнительную функциональность, например, проверку статуса сертификата (как OCSP) или получение промежуточных сертификатов (как CMP).

В [RFC 4387](#) определены некоторые ограниченные манипуляции с сертификатами X.509, ключами и списками CRL по протоколу HTTP с использованием метода GET (как раз этот метод мы использовали, когда последний раз покупали сертификат

Х.509). [RFC 4386](#) определяет использование DNS-записей [SRV](#) для обнаружения репозитория сертификатов и сервисов [OCSP](#).

Протокол управления сертификатами (Certificate Management Protocol, CMP)

Протокол управления сертификатами (CMP) определён в [RFC 4210](#) (обновлён в [RFC 6712](#)), формат сообщений запроса сертификатов (Certificate Request Message Format, CRMF) определён в [RFC 4211](#).

В дальнейшем описание будет расширено.

Протокол серверной валидации сертификатов (Server-Based Certificate Validation Protocol, SCVP)

В дальнейшем будет приведено описание. Варианты OCSP в значительной степени заменяют этот протокол.

Управление сертификатами поверх CMS (Certificate Management over CMS, CMC/CMS)

В дальнейшем будет приведено описание.

Протокол онлайн-получения статуса сертификата (Online Certificate Status Protocol, OCSP)

Протокол онлайн-получения статуса сертификата (OCSP) определён в [RFC 6960](#), а переработанный, обеспечивающий высокую производительность формат сообщений определён в [RFC 5019](#) (The Lightweight Online Certificate Status Protocol (OCSP) Profile for High-Volume Environments (облегчённый профиль OCSP для крупномасштабных сред)). В этом формате существенно уменьшен размер OCSP-запросов для одного сертификата и исключено большинство необязательных (OPTIONAL) полей в запросах и ответах (смотрите примечание ниже). RFC 6960 (отменяющий действие [RFC 2560](#) и [RFC 6277](#)) просто уточняет некоторые моменты в оригинальных RFC для достижения большей совместимости с [RFC 5019](#), а также добавляет свои собственные, совершенно новые туманные места в спецификацию. В [RFC 6961](#) определён новый формат 'certificate_request_v2', позволяющий серверам кэшировать (сохранять) ответы, а также позволяющий посылать в одном сообщении информацию о всех имеющих отношение к запросу сертификатах (включая промежуточные).

OCSP — это онлайн альтернатива [спискам отзыва сертификатов \(Certificate Revocation List, CRL\)](#). Если удостоверяющий центр поддерживает сервис OCSP, URI

этого сервиса обычно указывается в атрибуте [AuthorityInfoAccess](#) (AIA) сертификата (удостоверяющие центры, выпускающие сертификаты EV, обязаны поддерживать OSCP). Клиент отправляет запрос, опционально подписанный электронной подписью, идентифицирующий сертификат, который требуется проверить, и получает от сервера OSCP подписанный ответ, указывающий статус сертификата как действующий (good), отозванный (revoked) или неизвестный (unknown). В RFC разрешено передавать запросы и ответы посредством ряда различных транспортных протоколов (в частности, в качестве примеров упомянуты LDAP, SMTP и HTTP) с указанием конкретной транспортной схемы в URI из атрибута AIA сертификата, подвергающегося проверке. Также RFC определяет формат сообщений HTTP (используются GET и POST) при транспортировке OSCP. Формат сообщений запроса и ответа:

Формат запроса

OCSPRequest

TBSRequest

version	0 = v1
requestorName	OPTIONAL
requestList (один или несколько, в случае RFC 5019 - только один)	
reqCert	
hashAlgorithm	AlgorithmIdentifier
issuerNameHash	Хэш DN издателя
issuerKeyHash	Хэш открытого ключа издателя
serialNumber	CertificateSerialNumber
singleRequestExtensions	OPTIONAL
requestExtensions	OPTIONAL
optionalSignature	OPTIONAL

Примечания:

1. Подпись запроса OSCP является необязательной, и, если она присутствует, в поле **requestorName** должен быть указан субъект, подписавший сообщение. Очевидно, для проверки подписи у получателя сообщения должен быть открытый ключ субъекта, подписавшего сообщение (или его делегированного агента).
2. Поставщики сертификатов EV (Extended Validation) обязаны предоставлять сервис OSCP, для всех остальных типов сертификатов этот сервис является опциональным.

Формат ответа

OCSPResponse

responseStatus

successful	0	-- ответ содержит корректный статус
malformedRequest	1	-- неверный формат запроса
internalError	2	-- внутренняя ошибка сервера издателя
tryLater	3	-- повторите попытку позже
	-- (4)	не используется

```

sigRequired      5  -- запрос должен быть подписан
unauthorized     6  -- запрос неавторизован
responseBytes    OPTIONAL
responseType     OID (1.3.6.1.5.5.7.48.1.1 =BasicOCSPResponse)
  BasicOCSPResponse
    response
      ResponseData
        version      0 - версия, по умолчанию v1
        responderID  ОДНО ИЗ ДВУХ:
          byName      1 - имя
          byKey       2 - хэш открытого ключа издателя
        producedAt   GeneralizedTime
        responses
          certID
            hashAlgorithm  AlgorithmIdentifier
            issuerNameHash -- хэш DN издателя
            issuerKeyHash  -- хэш открытого ключа издателя
            serialNumber   CertificateSerialNumber
          certStatus      CertStatus
            good           0
            revoked        1
            unknown        2
          thisUpdate      GeneralizedTime
          nextUpdate      GeneralizedTime OPTIONAL
          singleExtensions OPTIONAL
          responseExtensions OPTIONAL
          signatureAlgorithm AlgorithmIdentifier,
          signature        Хэш данных ответа
          certs             OPTIONAL

```

Примечания:

1. Статус **good** в поле **certStatus** согласно RFC означает "не отозван", но в полях расширения (Extensions) может быть доступна дополнительная информация.
2. Статус **unauthorized** в поле **responseStatus** указывает на то, что сервер, выдающий ответ, не имеет достоверной информации по этому сертификату.
3. В [RFC 6960](#) определено несколько расширений для сообщений, кроме того, в них можно включать любые расширения [CRL \(RFC 2459\)](#).
4. Обычно ответ подписывается УЦ, издавшим сертификат, указанный в поле **serialNumber**, но протокол позволяет подписывать ответ делегированному удостоверяющему центру. В этом случае ответ должен включать сертификат (содержащий открытый ключ делегированного центра, подписавшего ответ) в поле **certs**, и этот сертификат должен быть подписан издателем сертификата, указанного в поле **serialNumber**.

5. [RFC 5019](#) определяет усовершенствованный формат сообщений (полностью совместимый с полным форматом OCSP) путём исключения большинства необязательных (OPTIONAL) полей с целью снижения нагрузки на сеть при передаче сообщений. Нововведения в протоколе: поддержка OCSP поверх HTTP только с использованием методов GET и POST. Нововведения в запросах: запрос ограничивается единственным сертификатом (значение поля **requestList** будет 1), обходится без необязательного поля **singleRequestExtensions**, необязательных структур **requestExtensions** и **optionalSignature** (если запрос подписан, отвечающий сервер вправе проигнорировать подпись). Нововведения в ответах: путём принудительного ограничения на один сертификат в запросе, RFC 5019 уже уменьшает сложность (и размер) ответа, кроме того, исключено необязательное поле **responseExtensions** (но поле **singleResponseExtensions** может быть включено). Опять же, если ответ подписан делегированным УЦ, он должен включать сертификат (содержащий открытый ключ делегированного центра, подписавшего ответ) в поле **certs**, и этот сертификат должен быть подписан издателем сертификата, указанного в поле **serialNumber**.

Узкие места OCSP

OCSP разрабатывался как система реального времени (в противоположность пакетной природе классических списков CRL), так что теоретически клиенты могут проверить статус любого сертификата до его принятия и использования. В частности, при обработке [сертификата EV](#) любая реализация клиента (например, браузер) в буквальном смысле обязана выполнить OCSP-проверку чтобы реализовать концепцию безопасности EV. Это означает, к примеру, что в результате каждого обращения к сервису HTTPS может (а в случае EV должна) происходить дополнительная проверка УЦ посредством обращения к сервису OCSP. Поскольку всё больше сайтов реализуют благонамеренную, но, по существу, ошибочную политику использования HTTPS для всего подряд (вместо того, чтобы настроить более значимый и надёжный сервис DNSSEC), производительность серверов OCSP резко снижается и они просто встают на колени от своего рода DDoS-атаки (хоть и из благих побуждений).

[RFC 6066](#) расширяет стандарт TLS, чтобы клиент мог запрашивать статус сертификата у сервера TLS (**OCSPStatusRequest**), тем самым, возможно, ещё более усугубляя проблему загрузки OCSP, упростив реализацию опроса OCSP и поощряя каждого клиента его выполнять. В [RFC 6961](#) предпринята попытка решить проблему загрузки OCSP путём использования нового запроса TLS 'certificate_request_v2', который, предположительно, позволяет (хотя при опросе OCSP сервер не обязательно должен придерживаться такого поведения) кэшировать ответы OCSP на сервере (снижая тем самым загрузку OCSP из-за проверок достоверности в режиме реального времени) и посылать несколько сообщений о статусе сертификатов

(включая все промежуточные сертификаты) в одном блоке. И наконец (мы на это надеемся, что это будет конец), поскольку многочисленные промежуточные сертификаты могут в сумме иметь серьёзный размер, в [RFC 7924](#) определён метод, с помощью которого клиент может сообщить серверу, что у него уже есть все необходимые для проверки промежуточные сертификаты.

Формат сертификата X.509

В этом разделе в убопомрачительных (но не всеобъемлющих) подробностях описывается формат и назначение основных полей (технически называемых атрибутами) сертификата X.509. Этот формат определён в [RFC 5280](#) (обновлён в [RFC 6818](#)).

Примечание: Хотя по текущим рекомендациям NIST (до 2030 года) длина ключа RSA должна быть 2048 бит, [RFC 8603](#) обязывает использовать RSA с ключами длиной 3072 или 4096 бит (либо ECDSA P-384) для сертификатов X.509, используемых в CNSA Suite (Commercial National Security Algorithm Suite).

Сертификат X.509 представляет собой структуру [ASN.1](#) (X.680), закодированную с помощью определённых в X.690 уникальных правил кодирования (Distinguished Encoding Rules, DER), и включающую в себя многочисленные ссылки на глобально уникальные [идентификаторы объектов \(Object Identifiers, OID\)](#).

Примечания:

1. В X.509 (и LDAP) при определении DN многие используют бестолковую псевдоВенгерскую нотацию (или lowerCamelCase, если Вы предпочитаете этот термин). В общем случае стоит помнить, что плохие реализации чувствительны к регистру символов, хорошие — нет. Более того, все правила соответствия LDAP, относящиеся к манипулированию DN, являются нечувствительными к регистру; это означает, что имена используемых в DN атрибутов нечувствительны к регистру.
2. [RFC 7250](#) определяет упрощённый формат сертификата для случаев, когда открытый ключ был получен в результате других доверенных процессов (возможно, офф-лайн). Этот минимальный формат сертификата использует только атрибут [subjectPublicKeyInfo](#) (и два его подполя). О том, что будет применяться данный минимальный формат сертификата, указывается в сообщениях ClientHello и ServerHello в ходе рукопожатия TLS/SSL.

Формальное определение сертификата X.509 даётся в нотации ASN.1 и выглядит следующим образом (выдержка из [раздела 14 RFC 5912](#)):

```
TBSCertificate ::= SEQUENCE {  
    version          [0] Version DEFAULT v1,
```

```

serialNumber      CertificateSerialNumber,
signature         AlgorithmIdentifier{SIGNATURE-ALGORITHM,
                  {SignatureAlgorithms}},
issuer            Name,
validity          Validity,
subject           Name,
subjectPublicKeyInfo SubjectPublicKeyInfo,
... ,
[[2:              -- If present, version MUST be v2
issuerUniqueID   [1]  IMPLICIT UniqueIdentifier OPTIONAL,
subjectUniqueID  [2]  IMPLICIT UniqueIdentifier OPTIONAL
]],
[[3:              -- If present, version MUST be v3 --
extensions       [3]  Extensions{{CertExtensions}} OPTIONAL
]], ... }

```

Кошмар! Если Вам требуется это понять (надеемся, что нет), можем порекомендовать это [руководство по выживанию в ASN.1 и DER](#). Предупреждение: прочтение приведённого там материала может негативно сказаться на вашем психическом здоровье.

Выводимый OpenSSL сертификат X.509 выглядит так (основные атрибуты описаны ниже):

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

bb:7c:54:9b:75:7b:28:9d

Signature Algorithm: sha1WithRSAEncryption

Issuer: C=MY, ST=STATE, O=CA COMPANY NAME, L=CITY, OU=X.509, CN=CA ROOT

Validity

Not Before: Apr 15 22:21:10 2008 GMT

Not After : Mar 10 22:21:10 2011 GMT

Subject: C=MY, ST=STATE, L=CITY, O=ONE INC, OU=IT, CN=www.example.com

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

RSA Public Key: (1024 bit)

Modulus (1024 bit):

00:ae:19:86:44:3c:dd...

...

99:20:b8:f7:c0:9c:e8...

38:c8:52:97:cc:76:c9...

Exponent: 65537 (0x10001)

X509v3 extensions:

X509v3 Basic Constraints:

CA:FALSE

Netscape Comment:

OpenSSL Generated Certificate

X509v3 Subject Key Identifier:

EE:D9:4A:74:03:AC:FB...
X509v3 Authority Key Identifier:
keyid:54:0D:DE:E3:37...

Signature Algorithm: sha1WithRSAEncryption
52:3d:bc:bd:3f:50:92...
...
51:35:49:8d:c3:9a:bb...
b8:74

Примечание: В приведённом сертификате строки, мало что дающие для понимания материала, были обрезаны или даже пропущены (места пропусков обозначены многоточиями).

Сертификат X.509 состоит из следующих полей (атрибутов):

Version	В общем случае этот атрибут будет указывать на версию 3 (X.509v3). Поскольку нумерация начинается с нуля, значение этого атрибута будет 2. Если атрибут опущен, подразумевается версия 1 (значение 0).
Serial Number	Положительное число размером максимум до 20 октетов. Определяет уникальный серийный номер каждого сертификата, выпущенного конкретным удостоверяющим центром (то есть сам по себе, в глобальном смысле, не являющийся уникальным) и использующийся, кроме всего прочего, в списках отзыва сертификатов (CRL) .
Signature	Значением данного атрибута должен быть тот же OID , что указан в атрибуте SignatureAlgorithm ниже.

Issuer

DN (Distinguished Name) удостоверяющего центра, подписавшего и, следовательно, издавшего данный сертификат. Это может быть корневой или некорневой удостоверяющий центр. Значение атрибута **issuer** может состоять из подмножества набора атрибутов domainComponent (DC=), countryName (C=), commonName (CN=), surname (SN=), givenName (GN=), pseudonym=, serialNumber=, title=, initials=, organizationName (O=), organizationalUnitName (OU=), stateOrProvinceName (ST=) и localityName (L=). Должны поддерживаться только атрибуты CN=, C=, ST=, O=, OU= и serialNumber=, остальные являются опциональными (serialNumber= встречается редко, но для сертификатов EV является обязательным, а L= встречается часто (хотя и трактуется неправильно), но в сертификатах EV используется как поясняющее значение). Теоретически, в [RFC 5280](#) разрешены любые глобально уникальные методы построения DN (формат X.500 O=, C= или формат IETF DC=, DC=), но де-факто все придерживаются формата X.500. DN издателя может выглядеть примерно так:

```
# Разбиение на две строки показано исключительно для
удобства
C=MY,ST=some state,L=some city,O=Expensive Certs inc,
OU=X.509 certs,CN=very expensive certs
# Существуют различные интерпретации атрибутов RDN,
# далее представлены общепринятые значения.
# C = двухбуквенный код страны согласно ISO3166
# ST = штат или область (край)
# L = местоположение, обычно - город
# O = организация - название компании
# OU = подразделение организации, обычно - тип
сертификата или бренд
# CN = общепринятое имя, обычно - наименование продукта
или бренд
```

Validity

Представляет собой два подполя (атрибута) **notBefore** и **notAfter**, определяющих период времени, в который сертификат является действительным, или, если присутствует только подполе **NotAfter**, его значение определяет тот момент, когда пользователю снова придётся раскошелиться удостоверяющему центру (издателю). Даты до 2049 года включительно кодируются как `UTCTime` (формат `YYMMDDHHMMSSZ`, да-да, год обозначается двумя цифрами), а после 2050 года — как `GeneralizedTime` (формат `YYYYMMDDHHMMSSZ`, год обозначается четырьмя цифрами). `Z` в формате представления времени — символьная константа, указывающая на использование часового пояса `Zulu Time` (UTC, он же время по Гринвичу), при её отсутствии подразумевается локальное время.

Subject

DN, определяющий субъекта, ассоциированного с этим сертификатом. Если это корневой сертификат, в атрибутах [issuer](#) и [subject](#) будут одинаковые DN (и в атрибуте [BasicConstraints](#) CA будет установлено в TRUE). В сертификате конечного пользователя/сервера DN в атрибуте subject тем или иным способом идентифицирует конечного субъекта. Как правило, атрибут CN (RDN) данного DN идентифицирует некоторое уникальное имя конечного субъекта, который будет сертифицирован или аутентифицирован. Значение атрибута **subject** может состоять из подмножества набора атрибутов domainComponent (DC=), countryName (C=), commonName (CN=), surname (SN=), givenName (GN=), pseudonym, serialNumber, title, organizationName (O=), organizationalUnitName (OU=), stateOrProvinceName (ST=) и localityName (L=). DN субъекта для аутентификации доступа к веб-сайту может выглядеть так:

```
# Разбиение на две строки показано исключительно для удобства
C=MY,ST=another state,L=another city,O=my
company,OU=certs,
CN=www.example.com
# Существуют различные интерпретации атрибутов RDN,
# далее представлены общепринятые значения.
# В случае персонального сертификата, могут встречаться
атрибуты
# GN=, SN= или pseudonym=
# C = двухбуквенный код страны согласно ISO3166
# ST = штат или область (край)
# L = местоположение, обычно - город
# O = организация - название компании
# OU = подразделение организации, обычно - подразделение
или наименование группы объектов
# CN = общепринятое имя, обычно - наименование конечного
субъекта, например www.example.com
```

Однако некоторые сертификаты используют DN из атрибута issuer и заменяют атрибут CN (RDN) именем аутентифицируемого субъекта. Исходя из такой логики, DN субъекта, сформированный на основании DN атрибута [issuer](#) из примера выше, будет выглядеть так:

```
# Разбиение на две строки показано исключительно для удобства
```

C=MY,ST=some state,L=some city,O=Expensive Certs inc,
OU=X.509 certs,CN=www.example.com

В приведённом примере CN=www.example.com будет срабатывать, если при доступе к веб-сайту используется URL <http://www.example.com>. Если же для доступа к этому сайту также используется <http://example.com>, процесс проверки сертификата завершится неудачей. В этом случае можно использовать атрибут [subjectAltName](#) для расширения сферы действия сертификата, чтобы включить example.com (или любые другие доменные имена, на которые распространяется действие этого сертификата). Такой сертификат многие поставщики называют SAN-сертификатом.

Хотя большинство сертификатов в настоящее время (2011 год) используют для описания субъекта RDN CN= атрибута **subject**, [RFC 6125](#) рекомендует использовать атрибут [subjectAltName](#), содержащий наименование конечного субъекта, и чтобы при валидации наименования субъекта значение атрибута **subjectAltName** имело бы приоритет перед значением атрибута CN= в атрибуте **subject**. Возможно, через некоторое время все приложения и издатели сертификатов будут придерживаться этих принципов, и потому, для обработки всех возможных вариантов, наименование субъекта следует помещать и в значение атрибута CN= атрибута **subject**, и в атрибуте **subjectAltName**.

Атрибут **subject** может быть пустым, в этом случае субъект, который будет аутентифицирован, определяется в атрибуте [subjectAltName](#).

Форма

CN=www.example.com/emailAddress=me@example.com встречается часто и обычно создаётся по умолчанию инструментами OpenSSL при генерации [запроса на подписание сертификата \(Certificate Signing Request, CSR\)](#). В этой форме определяется второй атрибут **emailAddress**, который может присутствовать или не присутствовать. Большинство удостоверяющих центров требуют, чтобы [строка с указанием адреса электронной](#)

[почты](#) оставалась пустой при создании CSR с помощью OpenSSL, возможно затем, чтобы продать Вам ещё один сертификат для защиты адресов электронной почты. Или это кажется Вам слишком циничным? [Замечания по атрибутам сертификата subject и subjectAltName](#).

SubjectPublicKeyInfo	Содержит два подполя (атрибута), algorithm (OID алгоритма открытого ключа из списка, определённого в RFC 3279) и subjectPublicKey (открытый ключ субъекта в виде строки бит). Пример атрибута algorithm при использовании OID алгоритма RSA (rsaEncryption): 1.2.840.113549.1.1.1 Примечание: В RFC 7250 определён упрощённый формат сертификата, состоящий только из данного атрибута и двух его подполей.
IssuerUniqueIdentifier	Необязательный атрибут. Определяет уникальное значение для издателя. В RFC рекомендуется, чтобы этот атрибут НЕ присутствовал.
SubjectUniqueIdentifier	Необязательный атрибут. Определяет уникальное значение для субъекта, который будет аутентифицирован. В RFC рекомендуется, чтобы этот атрибут НЕ присутствовал.

Великое множество расширений определено в различных RFC, далее представлены только самые значимые (по нашему скромному мнению). Расширения могут быть помечены как критичные (CRITICAL).

Примечание: Пометка CRITICAL имеет интересную и тонкую интерпретацию. Если обрабатывающее сертификат программное обеспечение видит значение с пометкой CRITICAL (которую оно всегда может интерпретировать), но не может распознать данное расширение, оно ДОЛЖНО отказаться от обработки и НЕ принимать этот сертификат.

Примечания:

1. В [RFC 5280](#) даны определения стандартных расширений (OID которых начинается на 2.5.29, определены в оригинальных стандартах X.509), а также так называемых "частных расширений" (Private Extensions) (OID которых начинается на 1.3.6.1.5.5.7.1). Распределение номеров для расширений из пространства OID Private Internet поддерживается [IANA](#). Приведённые ниже расширения, если это не оговаривается отдельно, являются стандартными (OID из пространства 2.5.29).
2. В [RFC 7633](#) определено расширение сертификата X.509 из пространства Private Internet (OID 1.3.6.1.5.5.7.1.24 id-pe-tlsfeature), позволяющее включать в сертификат расширения функционала TLS (идентифицирующиеся по значениям TLS-кодов). По сути, такие расширения могут позволить клиенту выявить потенциально мошеннический сервер. Так, если сертификат содержит расширение функционала TLS status_request, а сервер не возвращает информации о статусе (в сообщении **CertificateStatus**), то клиент может сделать вывод о потенциальной небезопасности сессии.

AuthorityInfoAccess Расширение из пространства Private Internet (OID 1.3.6.1.5.5.7.1.1). Расширение широко известно по аббревиатуре AIA (не путайте с AIA в LDAP — это разные вещи). Используется для хранения информации о сервисах УЦ, в том числе [Online Certificate Status Protocol \(OCSP\)](#) любых форматов. Интересно, что хотя [сертификаты EV](#) требуют, чтобы служба определения статуса сертификата работала в режиме 24x7 (и чаще всего такая служба реализуется с помощью OCSP), в действующих стандартах EV использование этого атрибута не рассматривается.

```
AuthorityInfoAccess = AuthorityInfoAccessSyntax
# могут существовать несколько элементов
AuthorityInfoAccessSyntax,
# каждый состоит из:
    accessMethod = OID
# может принимать значения:
# 1.3.6.1.5.5.7.48.1 = ocsp
# accessLocation = URI сервиса OCSP УЦ
# для УЦ-издателя
# 1.3.6.1.5.5.7.48.2 = caIssuers
# используется только если сертификат подписан не
корневым УЦ
# accessLocation = URI описания корневого УЦ

    accessLocation = URI (исключительно адрес
электронной почты
                                или X.500 DirectoryString)
```

authorityKeyIdentifier [OID: 2.5.29.35] Необязательное расширение.

Может содержать три атрибута:

keyIdentifier	[0] KeyIdentifier
authorityCertIssuer	[1] GeneralNames
authorityCertSerialNumber	[2] CertificateSerialNumber

Стандарт рекомендует использовать значение **keyIdentifier** для всех сертификатов, кроме корневого. Обычно **keyIdentifier** — это 160-битный хэш SHA-1 атрибута [subjectPublicKeyInfo](#), но определены и другие методы. Наличие этого атрибута упрощает создание пути (цепочки) сертификатов, позволяет удостоверяющим центрам иметь несколько корневых сертификатов, у каждого из которых может быть разный ключ, на который ссылается это расширение.

subjectKeyIdentifier [OID: 2.5.29.14] Необязательное расширение, но стандарт рекомендует использовать это значение во всех сертификатах в качестве вспомогательного средства для построения пути (цепочки) сертификатов. Обычно **SubjectKeyIdentifier** — это 160-битный хэш SHA-1 атрибута [subjectPublicKeyInfo](#), но определены и другие методы.

KeyUsage	[OID: 2.5.29.15] Битовая строка, определяющая цели, для которых может использоваться открытый ключ. Может принимать следующие значения: - digitalSignature (0) - nonRepudiation (1) - keyEncipherment (2) - dataEncipherment (3) - keyAgreement (4) - keyCertSign (5) # указывает на то, что это сертификат УЦ - cRLSign (6) - encipherOnly (7) - decipherOnly (8) Если задано значение keyCertSign, то в расширении BasicConstraints значение cA ДОЛЖНО БЫТЬ также установлено в TRUE, однако, если присутствует расширение BasicConstraints, в котором cA установлено в TRUE, то наличия KeyUsage keyCertSign не требуется.
----------	---

ExtendedKeyUsage [OID: 2.5.29.37] Если это расширение присутствует, в нём уточняются цели, для которых может быть использован открытый ключ. Значение этого атрибута должно соотноситься со значением атрибута **KeyUsage**. Оно может быть одним из следующих:

serverAuth (1) TLS-аутентификация сервера WWW
(действительно с digitalSignature,
keyEncipherment или keyAgreement)
clientAuth (2) TLS-аутентификация клиента WWW
(действительно с digitalSignature или
keyAgreement)
codeSigning (3) Подписание доступного для
скачивания исполняемого кода
(действительно с digitalSignature)
emailProtection (4) Защита электронной почты
(действительно с digitalSignature,
nonRepudiation,
и/или (keyEncipherment или keyAgreement))
timeStamping (8) Привязка хэша объекта ко времени
(действительно с digitalSignature и/или
nonRepudiation)
OCSPSigning (9) Подписание ответов OCSP
(действительно с digitalSignature и/или
nonRepudiation)

Basic Constraints [OID: 2.5.29.19] Логическое значение, определяющее, является ли данный сертификат корневым сертификатом (сертификатом УЦ) (TRUE) или нет (FALSE). В дополнении может также присутствовать необязательный атрибут pathLenConstraint, определяющий максимальную глубину цепочки сертификатов.

сА TRUE | FALSE
pathLenConstraint INTEGER

CRL Distribution
Points

[OID: 2.5.29.31] Необязательное, но РЕКОМЕНДУЕМОЕ RFC расширение (попробуй разберись). Определяет одно или несколько URL (и другую опциональную информацию), по которым можно получить [списки отзыва сертификатов \(CRL\)](#) для удостоверяющего центра, выпустившего данный сертификат (**issuer**). Каждое расположение CRL, называемое DistributionPoint (пункт распространения), имеет следующий формат:

```
# (0) = OPTIONAL
distributionPoint DistributionPointName (0)
# points to a structure defined below
reasons ReasonFlags (0)
cRLIssuer GeneralNames (0)
# DN издателя CRL, если он не совпадает с
# издателем сертификата.
# Данное DN может использоваться в поисковых
запросах LDAP

# DistributionPointName может быть ЛИБО
fullName GeneralNames
# может содержать URI,
# например, http://crl.example.com,
# или название протокола, например, HTTP, LDAP,
FTP и т.п.
# используемого для получения CRL
# ЛИБО
nameRelativeToCRLIssuer
# RDN, добавляемое к DN издателя для получения
CRL

# ReasonFlags может принимать одно из следующих
значений:
unused 0
keyCompromise 1
cACompromise 2
affiliationChanged 3
superseded 4
cessationOfOperation 5
certificateHold 6
privilegeWithdrawn 7
aACompromise 8
```

Разрешается наличие нескольких подобных записей.

CertificatePolicies

[OID: 2.5.29.32] Необязательное расширение. Может быть использовано для идентификации конкретных политик удостоверяющего центра [issuer](#). Данное расширение позволяет указывать как OID (в **CertPolicyId**), так и другие опциональные атрибуты, которые, по существу, содержат (ссылаются на) читабельный текст, но RFC РЕКОМЕНДУЕТ использовать только OID. OID в данном атрибуте также используется для идентификации [сертификата EV](#). Значение может быть любым из приведённых ниже атрибутов:

policyIdentifier	CertPolicyId # OID
policyQualifiers	# разрешено несколько значений
	# обычно представляет собой URI
на текстовое	
	# изложение политики или просто
текст	

subjectAltName [OID: 2.5.29.17] Иногда сокращённо обозначается как SAN. Необязательное расширение, но [RFC 6125](#) рекомендует, чтобы в сертификатах серверов этот атрибут всегда присутствовал и в нём содержалось имя субъекта, для которого выпущен сертификат (обосновывается это тем, что атрибут **dNSName** этого расширения как раз и был определён для хранения имени сервера, а в атрибуте CN= атрибута **subject** может содержаться информация любого формата). Расширение может также быть использовано для определения дополнительных субъектов, охватываемых действием данного сертификата (атрибут [subject](#) не пуст), либо как альтернатива атрибуту [subject](#) (атрибут subject пуст, а расширение **subjectAltName** помечено как CRITICAL). Значение может быть любым из приведённых ниже атрибутов:

otherName	Пары type=, value= включая имена Kerberos
(RFC 4556):	
	oid = 1.3.6.1.5.2.2 kerberos-principal
(IA5String)	ИЛИ
	SRVName (RFC 4985):
	oid =
1.3.6.1.5.5.7.8.7	srv-name (IA5String)
rfc822Name	email me@example.com
dNSName	DNS-имя
host1.example.com	
x400Address	Почтовый адрес X.400, если Вы в курсе, что
это	
directoryName	Альтернативный DN
ediPartyName	Материалы EDI
uniformResourceIdentifier	URI
ldap://ldap.example.com	
iPAddress	IP V4/V6 192.168.0.1
registeredID	OID

Разрешается присутствие нескольких записей. Использование **subjectAltName** — путь решения проблемы нескольких имён DNS у одного сервера. Например, если доступ к серверу осуществляется

с использованием <https://www.example.com> и <https://example.com>, то www.example.com может присутствовать в атрибуте CN= атрибута subject, а example.com может присутствовать в атрибуте dNSName этого расширения, или, чаще, оба имени www.example.com и example.com будут присутствовать в атрибутах dNSName.

Примечание: в этих атрибутах могут быть любые доменные имена, например, сразу и www.example.com, и www.example.net. Требование общего корня доменного имени не выдвигается. Не все удостоверяющие центры поддерживают использование **subjectAltName**. Генерация записей subjectAltName в самоподписанных сертификатах средствами OpenSSL — несколько грубоватый процесс, мы подробно описали его [ниже](#). [Замечания по атрибутам сертификата subject и subjectAltName](#).

SignatureAlgorithm Алгоритм (идентифицируемый по [OID](#)), используемый для подписания сертификата, и любые ассоциированные с ним параметры. Например, OID цифровой подписи RSA с SHA1 (sha1WithRSAEncryption):

1.2.840.113549.1.1.5

Это значение должно совпадать с указанным в атрибуте [signature](#) сертификата. Список действительных для использования с сертификатами X.509 OID определён в [RFC 3279](#).

SignatureValue Битовая строка, содержащая цифровую подпись.

Все атрибуты, за исключением **SignatureAlgorithm** и **SignatureValue** закодированы с помощью правил кодирования ASN.1 DER (Distinguished Encoding Rules), определённых в стандарте X.690 ITU-T. Цифровая подпись охватывает все закодированные DER элементы, кроме, очевидно, самой себя.

Некоторые организации, занимающиеся стандартизацией (национальные, промышленные организации или правительственные агентства) определяют

специфичные профили, в которых уточняется значение некоторых атрибутов, либо определяются специфичные атрибуты. Всё очень запутанно.

Замечания по атрибутам сертификата **subject** и **subjectAltName**

Сертификаты X.509 используются в различных целях. В этом разделе описываются ограничения и дополнительные правила, используемые (налагаемые) для обеспечения некоторых функциональных возможностей.

Проверка доменного имени по сертификатам X.509

В [RFC 6125](#) определяется набор правил, цель которых — уменьшить путаницу между клиентами, серверами и УЦ, выдающими сертификаты, по вопросу того, как следует описывать или как могут быть описаны конечные субъекты в атрибутах **subject** и **subjectAltName**. (В [RFC 6186](#) описан способ обнаружения хоста электронной почты с помощью ресурсных записей [SRV](#) DNS, а в [RFC 7817](#) разъясняются требования [RFC 6125](#) в отношении протоколов, связанных с электронной почтой). Исторически конечные субъекты определялись в RDN CN= атрибута **subject**, и хотя это до сих пор поддерживается (в основном по историческим соображениям), в [RFC 6125](#) предпочтительным названо использование атрибута **subjectAltName**, позволяющего обеспечить большую гибкость и ясность. [RFC 6125](#) определяет четыре возможности для проверки конечной системы:

Примечание: Во всех случаях при использовании **subjectAltName** может присутствовать более одного типа имени. Кроме того, может быть определено более одного субъекта для каждого типа имени. Конечные субъекты, описанные во всех присутствующих типах, образуют совокупность конечных субъектов данного сертификата. В случае использования атрибута **subject** действие сертификата распространяется только на одного конечного субъекта, описанного в RDN CN= (хотя в этом случае также может присутствовать атрибут **subjectAltName**).

1. В атрибуте **subjectAltName** содержится атрибут типа **dNSName**. В этом случае указанное имя представляет собой имя конечного субъекта, на которого распространяется действие данного сертификата.

В [RFC 7817](#) сказано, что для электронной почты в качестве конечного субъекта следует указывать либо доменную часть адреса электронной почты (RFC822), к примеру, для адреса `user@example.com` конечным субъектом будет `example.com`, и/или в качестве конечного субъекта может быть указано FQDN хоста, получающего электронную почту, например, `mail.example.com`.

2. В атрибуте **subject** содержится единственное значение атрибута CN, например, **cn=hostname** (могут также присутствовать другие RDN, но при проверке доменного имени они игнорируются). В этом случае **hostname** представляет собой имя конечного субъекта, на которого распространяется действие данного сертификата.

В [RFC 7817](#) сказано, что для электронной почты в качестве конечного субъекта следует указывать либо доменную часть адреса электронной почты (RFC822), к примеру, для адреса user@example.com конечным субъектом будет example.com, и/или в качестве конечного субъекта может быть указано FQDN хоста, получающего электронную почту, например, mail.example.com.

3. В атрибуте **subjectAltName** содержится атрибут типа **otherName**, а значение type самого атрибута otherName установлено в SRV (oid = 1.3.6.1.5.5.7.8.7) (определено в [RFC 4985](#)).

В [RFC 6186](#) определён порядок использования ресурсных записей DNS SRV для нахождения сервисов, связанных с электронной почтой (формат довольно странный). Эти рекомендации используются в [RFC 7817](#).

4. В атрибуте **subjectAltName** содержится атрибут типа **uniformResourceIdentifier**. В этом случае тип сервиса определяется явно.

Списки отзыва сертификатов (Certificate Revocation Lists, CRL)

Списки отзыва сертификатов (CRL) — метод, с помощью которого сертификаты могут быть признаны недействительными до наступления даты, указанной в атрибуте [NotAfter](#) сертификата. Обычно CRL выпускаются тем же УЦ, который выпустил и сам сертификат. CRL могут быть получены различными методами, например, посредством LDAP, HTTP или FTP. Формат CRL (в частности CRLv2, текущая версия) определён в [RFC 5280](#), а затем обновлён в [RFC 6818](#).

Теоретически, при получении сертификата от сервера, программное обеспечение, получившее сертификат, должно проверить, что этот сертификат отсутствует в CRL (не был отозван). Чтобы свести к минимуму возможность ошибочного приёма отозванного сертификата, каждый раз при получении сертификата от сервера должна быть получена последняя версия CRL. Поскольку CRL содержит список всех отозванных сертификатов какого-либо УЦ, он может быть внушительных размеров и создавать, таким образом, неприемлемую избыточность трафика при первоначальном соединении TLS/SSL. Кроме того, в зависимости от политики удостоверяющего центра, CRL могут обновляться каждый час, 12 часов, раз день и т.д. Суть в том, что полученные файлы CRL, даже если мы запрашиваем их снова и

снова при каждом получении сертификата, всё равно могут содержать просроченные данные.

На практике получение CRL каждый раз при проверке сертификата выполняется довольно редко (если вообще выполняется), многие системы полагаются на кэши CRL, которые периодически обновляются. В RFC 5280 для описания частоты обновления CRL используется обтекаемый термин **suitably-recent (достаточно свежее)**. Онлайн версия списков отзыва, известная как [Online Certificate Status Protocol \(OCSP\)](#), определена в [RFC 2560](#), переработана в [RFC 5019](#) и обновлена в [RFC 6277](#). Формат CRL:

Version	Необязательный атрибут. В общем случае этот атрибут будет указывать на версию 2 (CRLv2). Поскольку нумерация начинается с нуля, значение этого атрибута будет 1. Если атрибут опущен, подразумевается версия 2 (значение 1).
Signature	Значением данного атрибута должен быть тот же OID, что указан в атрибуте SignatureAlgorithm ниже.
Issuer	DN удостоверяющего центра, подписавшего и, следовательно, издавшего данный CRL. Это может быть корневой или некорневой удостоверяющий центр. Смотрите также описание атрибута issuer сертификата.
ThisUpdate	Время и дата создания CRL. Формат даты может быть UTCTime (YYMMDDHHMMSSZ) или GeneralizedTime (YYYYMMDDHHMMSSZ). Z в формате представления времени — символьная константа, указывающая на смещение относительно часового пояса Zulu Time (UTC, он же время по Гринвичу (GMT)), при её отсутствии подразумевается локальное время.
NextUpdate	Время и дата создания следующего CRL. Формат даты может быть UTCTime (YYMMDDHHMMSSZ) или GeneralizedTime (YYYYMMDDHHMMSSZ). Z в формате представления времени — символьная константа, указывающая на смещение относительно часового пояса Zulu Time (UTC, он же время по Гринвичу (GMT)), при её отсутствии подразумевается локальное время.

RevokedCertificates	В CRL отозванные сертификаты идентифицируются по их серийным номерам. Серийные номера не глобально уникальны, но уникальность поддерживается в серийных номерах сертификатов, выпущенных одним УЦ, таким образом, для получения уникальности клиенты должны использовать комбинацию серийного номера и атрибута issuer. userCertificate revocationDate crlEntryExtensions CertificateSerialNumber Time Extensions OPTIONAL
Extensions	Несколько расширений CRL определены в RFC 3280, ни одно из них не помечено как критичное (CRITICAL) и многие из них могут присутствовать лишь в случае непрямого CRL, то есть CRL, выпущенного сторонним УЦ (не тем УЦ, который выпустил отзываваемый сертификат).
SignatureAlgorithm	Алгоритм (идентифицируемый по OID), используемый для подписания сертификата, и любые ассоциированные с ним параметры. Например, OID цифровой подписи RSA с SHA1 (sha1WithRSAEncryption): 1.2.840.113549.1.1.5 Это значение должно совпадать с указанным в атрибуте signature CRL. Список действительных для использования с сертификатами X.509 OID определен в RFC 3279.
SignatureValue	Битовая строка, содержащая цифровую подпись.

Процесс доверия — удостоверяющие центры и сертификаты X.509

Как было сказано выше, сертификат X.509 является доверенным источником открытого ключа субъекта, указанного в сертификате (обычно указывается в атрибуте CN атрибута [subject](#) или в дополнении [subjectAltName](#)). Доверие формируется во время процесса создания сертификата. В зависимости от УЦ, процесс получения сертификата X.509 будет различаться в деталях, но, в общем случае, он состоит из следующих шагов:

1. Первое (самого высокого уровня) звено в цепи доверия — это удостоверяющий центр (УЦ). УЦ считаются доверенными организациями либо потому, что они сами так заявляют, либо потому, что они удовлетворяют каким-либо стандартам. Самой признанной организацией в сфере стандартизации УЦ в Северной Америке является [WebTrust](#), и большинство УЦ указывают в документации, что они подвергались ревизии аккредитованным аудитором WebTrust (хотя в последнее время всё чаще подобные проверки выполняют и другие крупные международные аудиторские конторы). Сам факт основания УЦ является первым шагом в создании линии доверия. В какой-то момент времени УЦ генерирует одну или несколько пар асимметричных ключей и с помощью закрытого ключа выпускает [самоподписанный сертификат](#) (атрибуты [issuer](#) и [subject](#) данного сертификата X.509 совпадают и в расширении [BasicConstraints](#) сА установлен в TRUE). Этот сертификат является корневым сертификатом удостоверяющего центра, а закрытый ключ, парный к которому открытый ключ содержится в корневом сертификате, используется для подписания пользовательских сертификатов. Корневые сертификаты (или сертификаты УЦ) распространяются среди конечных пользователей посредством какого-либо доверенного процесса, чаще всего с инсталлятором браузера.
2. Пользователь, желающий получить сертификат, рассматривает продукты различных удостоверяющих центров (УЦ) и выбирает наиболее подходящий для него УЦ. Выбранному УЦ (или одному из его агентов — регистрационных центров (РЦ)) подаётся заявка на конкретный тип сертификата SSL (X.509). В зависимости от типа сертификата затребовывается и (как правило) проверяется различная информация, такая как наименование бизнеса, регистрационный номер бизнеса или иная идентификационная информация. Опять же, в зависимости от типа сертификата, могут быть затребованы дополнительные подтверждения идентичности. В основном вся эта информация подвергается ручной обработке (хотя процесс может быть в большей или меньшей степени автоматизирован). В результате этих проверок устанавливается следующее звено в цепи доверия.

Удостоверяющий центр, сам по себе являющийся доверенным, уполномочен достоверно (доверенно) заявить о том (установить то), что пользователь является именно тем, за кого он себя выдаёт. В большей или меньшей степени.

3. После выбора продукта SSL, предоставления необходимой идентификационной информации и проверки её удостоверяющим центром, пользователю предлагается сгенерировать набор асимметричных ключей и с помощью закрытого ключа подписать запрос на подписание сертификата (Certificate Signing Request, CSR), в котором, кроме прочей информации, будет содержаться открытый ключ из сгенерированной пары ключей. Формат CSR определён в [RFC 2986](#) (перепубликация стандарта RSA, PKCS#10, обновлён в [RFC 5967](#)) и содержит следующие данные:

Version	Version 0.
Subject	DN, определяющее субъекта, который будет ассоциирован с сертификатом. В общем случае, в зависимости от политик УЦ, это должно быть то значение, которое будет помещено в атрибут subject возвращаемого сертификата X.509.
SubjectPublicKeyInfo	Содержит два подполя (атрибута): algorithm (OID алгоритма шифрования с открытым ключом из списка, определённого в RFC 3279) и subjectPublicKey (открытый ключ субъекта в виде битовой строки). Пример OID алгоритма RSA (rsaEncryption): 1.2.840.113549.1.1.1
Attributes	Необязательный атрибут. Может содержать несколько подполей (атрибутов), пока определены следующие: challengePassword (пароль для дополнительной защиты CSR — большинство УЦ настаивают, чтобы этот атрибут НЕ присутствовал) и unstructuredName (любой подходящий текст — опять же, обычно УЦ не требуют его наличия или игнорируют, если таковой имеется).

SignatureAlgorithm	Алгоритм (идентифицируемый по OID), используемый для подписания CSR, и любые ассоциированные с ним параметры. Например, OID цифровой подписи RSA с SHA1 (sha1WithRSAEncryption):
--------------------	---

1.2.840.113549.1.1.5

Примечания:

1. CSR подписывается с помощью закрытого ключа из пары закрытого и открытого ключей, соответствующий открытый ключ помещается в атрибут **SubjectPublicKeyInfo** этого CSR.
2. [Расширения сертификата](#), которые будут присутствовать в итоговом сертификате, могут также указываться, если этот сертификат будет самоподписанным. Некоторые коммерческие УЦ поддерживают расширения.

SignatureValue	Битовая строка, содержащая цифровую подпись.
----------------	--

Процедура создания CSR описана [далее](#).

4. CSR загружается на сервер УЦ (обычно по FTP или HTTP). УЦ использует данные из CSR и, возможно, другую информацию, полученную из заявки на сертификат SSL, для создания сертификата X.509 пользователя, обычно со сроком действия от 1 до 3 лет. Наконец, УЦ подписывает сертификат пользователя (атрибуты [SignatureAlgorithm](#) и [SignatureValue](#)) используя закрытый ключ, парный к которому открытый ключ содержится в корневом сертификате УЦ. Сертификат X.509 тем или иным способом (FTP/HTTP/EMAIL) отправляется пользователю.
5. Таким образом, цикл доверия завершается цифровой подписью удостоверяющего центра. Цифровая подпись пользовательского сертификата может быть верифицирована только с помощью открытого ключа издателя (атрибут [issuer](#)), который содержится в корневом сертификате УЦ, получаемом путём [какого-либо \(доверенного\) процесса](#) (обычно с инсталлятором браузера).

При получении браузером сертификата от веб-сайта (во время протокола рукопожатия TLS/SSL) должен быть проверен весь путь вплоть до корневого сертификата (сертификата УЦ). На этом пути может быть один или несколько уровней сертификатов, в зависимости от поставщика сертификата. Корневые

сертификаты (сертификаты УЦ), а также какие-либо необходимые промежуточные сертификаты, обычно распространяются в составе программного обеспечения основных браузеров. Распространяемые таким способом корневые сертификаты (сертификаты УЦ) часто называются якорями доверия — широко распространённый термин, используемый для описания любой базовой структуры или информации, полученной путём доверенного распространения, с помощью которой можно проверить/аутентифицировать полученную информацию. В [RFC 5914](#) (и немного в [RFC 5937](#)) определено, каким образом могут быть организованы, использованы и обработаны такие якоря доверия в конкретных случаях применения сертификатов X.509/SSL.

Сертификаты EV

Сертификаты расширенной валидации (Extended Validation, EV) определены [CA/Browser Forum](#) и включают в себя сочетание продвинутых методов канцелярских проверок и технических процессов. Цель сертификатов EV — обеспечение повышенного доверия к конечным пользователям, хотя в стандартах прямо заявляется, что эти сертификаты не гарантируют, что владелец сертификата занимается заявленной деятельностью (бизнесом) — только то, что владелец действительно существует.

В браузерах, поддерживающих сертификаты EV, при защищённом соединении пользователь видит обычный значок замка, но при применении сертификата EV строка состояния подсвечивается зелёным цветом. В настоящий момент сертификаты EV поддерживают следующие браузеры: MSIE (только 7) и последние версии Konqueror, Firefox (v3+) и Opera(9.5+). Обычно сертификаты EV значительно дороже, нежели другие типы сертификатов. Характеристики стандарта издания сертификатов EV:

1. Удостоверяющий центр должен быть аттестован на соответствие EV и подвергаться ежегодной переаттестации.
2. При расширенной проверке пользователя, кроме всего прочего, требуется, чтобы УЦ убедился, что пользователь, подавший заявку, действительно является владельцем того доменного имени, аутентификацию которого он стремится получить!
3. Удостоверяющие центры следуют практикам, изложенным в [RFC 3647](#), который имеет статус информационного (INFORMATIONAL), то есть все остальные могут его не придерживаться.

4. Стандартизируется использование и значение некоторых атрибутов DN в атрибуте [subject](#), а также добавляются несколько новых. В частности, следующие атрибуты определены как требуемые (REQUIRED) — эквивалент ключевого слова MUST из RFC в стандартах EV:
1. O - organizationalName (OID 2.5.4.10). Полное юридическое название организации пользователя.
 2. businessCategory (OID 2.5.4.15). Определяет, какие категории сертификатов могут подойти, могут быть разделы 5b, 5c или 5d.
 3. C - Country - (OID 2.5.4.6), ST - stateOrProvinceName - (OID 2.5.4.8) и L - localityName - (OID 2.5.4.7). Значения всех этих атрибутов должны соответствовать легальной юрисдикции бизнес-субъекта (а не, скажем, расположению веб-сервера).
 4. serialNumber (OID 2.5.4.5). Регистрационный номер бизнеса.
 5. CN - CommonName - (OID 2.5.4.3). Доменное имя хоста, например, [www.example.com](#).
5. Обязательное требование: предоставление EV-удостоверяющими центрами онлайнowych (24x7) возможностей проверки статуса любого сертификата EV. Обычно это достигается путём использования [Online Certificate Status Protocol \(OCSP\)](#) ([RFC 2560](#)).
6. Обязательное требование: сертификаты EV могут выдаваться только для аутентификации серверов (по крайней мере пока), то есть в атрибуте CN= должно быть имя сервера, такое как [www.example.com](#) или [mail.example.com](#).
7. Точное указание на то, что это именно сертификат EV, производится путём использования зарегистрированного OID (уникального для каждого УЦ) в атрибуте [CertificatePolicies](#).

Создание сертификатов и CSR с помощью OpenSSL

[Создание самоподписанных сертификатов \(OpenSSL, несколько методов\)](#)

Замечания по форматам файлов, связанных с SSL

В настоящее время существует множество форматов файлов, используемых для хранения сертификатов, ключей и других данных, связанных с X.509/SSL. Иногда расширения таких файлов дают представление об их содержимом, иногда — нет. Прежде чем углубляться в детали, прочтите этот краткий обзор:

1. Все связанные с SSL объекты (сертификаты, ключи и прочие) изначально используют кодировку DER. DER — это бинарная (8 бит) кодировка. Это, кроме всего прочего, означает, что пересылать такие данные по электронной почте нельзя. [PEM \(Privacy Enhanced Mail\)](#) — это метод простого перекодирования (с использованием base64) первоначальных форматов DER в нечто такое, что может быть отправлено по электронной почте или передано по другим коммуникационным системам.
2. Некоторые [стандарты PKCS#X](#), в частности, PKCS#7 (и его CMS-эквивалент от IETF [RFC 5652](#)), PKCS#12 и PKCS#8, представляют собой контейнеры (закодированные в формат DER), используемые для определения нескольких объектов в одном и том же файле. Если файл содержит один объект, скажем, сертификат или CRL, то такому объекту контейнер не требуется (хотя, опционально, он может быть и заключён в контейнер).
3. По своей сути, единичный ключ выглядит как один объект, и потому контейнер ему не требуется. Однако, помимо самого ключа, требуется также информация об алгоритме его использования (а также другие параметры), следовательно, для единичного ключа всё-таки требуется контейнер для определения составляющих объектов этого ключа (в данном случае [PKCS#8](#)). И наоборот, сертификат X.509v3 содержит много различной информации, и потому может показаться, что для него требуется контейнер. Но сертификат X.509v3 сам по себе является контейнером. Поэтому, если в файле содержится только единичный сертификат, то он не нуждается в контейнере (например, файлы с расширениями .cer или .crt). Однако, когда в одном файле содержится сертификат и закрытый ключ (файлы с расширениями .p12/.pkx), то в этом случае присутствуют как минимум два объекта, требующие определения, и потому здесь контейнер необходим (в данном случае [PKCS#12](#)).
4. Многие форматы из [стандартов PKCS#X](#) представляют собой контейнеры (закодированные в DER), содержимое которых по своей природе предназначено для передачи посредством какой-либо системы связи, например, запросы на подписание сертификата (CSR). В таких случаях сформированные данные, независимо от расширения файла, в конечном итоге кодируются в PEM.
5. Во многих случаях содержимое файла определяется контекстом ситуации, а не расширением файла. Так, если ожидается, что файл должен содержать только сертификат (без закрытого ключа), то он может иметь расширения как .pem, так и .crt, и даже .cer.
6. В качестве простейшего теста откройте файл, независимо от его расширения, в текстовом редакторе. Если там какая-то абракадабра, значит он закодирован в DER. Если Вы можете прочитать '-----BEGIN' — это PEM.

Формат PEM

В качестве формата по умолчанию (основного формата) OpenSSL поддерживает формат почты повышенной конфиденциальности Privacy Enhanced Mail (PEM). Изначальный формат сертификатов X.509, как, впрочем, и всех остальных объектов, связанных с SSL, — ASN.1 DER (бинарный формат). PEM кодирует бинарный блок DER в base64 (RFC 3548), при этом создаётся текстовая версия (подмножество набора символов ASCII/IA5) которая может, кроме всего прочего, пересылаться почтовыми системами. Закодированные в PEM объекты включают заголовочную и завершающую строки, каждая из которых начинается и заканчивается ровно пятью дефисами и представляет собой пригодное для чтения людьми описание того содержимого в формате base64, которое заключено между этими строками. Файлы PEM выглядят примерно так:

```
-----BEGIN CERTIFICATE-----
MIIDHDCCAoWgAwIBAgIJALt8VJ...
...
Cfh/ea7F1El1Ym1Zj2v3wLhRl1...
NH5lEmZybl+m2frlkjUv9KAavxc...
IFgovdU8YPMDds=
-----END CERTIFICATE-----

BLAH BLAH BLAH

-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4, ENCRYPTED
DEK-Info: DES-EDE3-CBC, 6EF6203EF1A9533A

r7LMq15wr100mMsD84KyNo+5yY...
El3/msvQ98BkaMihajEn5f2Ux0...
...
f6uoSk8HBZLI tWtxqRuBRVb8jq...
hdp9hvv dja9XI rAPGQJ0u2QVw==
-----END RSA PRIVATE KEY-----
```

Примечание: В приведённом примере строки, мало что дающие для понимания материала, были обрезаны или даже пропущены (места пропусков обозначены многоточиями).

Текст BEGIN CERTIFICATE и END CERTIFICATE из примера может принимать различные значения, описывающие функциональность и формат материала, содержащегося между строками-ограничителями. В любом файле PEM может быть более одного элемента (каждый из которых ограничен последовательностью -----BEGIN -----END), а также этот файл может содержать другую информацию до, после и между такими элементами, но не внутри ограничителей. Формат PEM определён в [RFC 1421](#) для использования с S/MIME ([RFC 3850](#)).

Ключевые слова, используемые в элементах BEGIN формата PEM

В [RFC 7468](#) определено несколько ключевых слов (или меток), которые могут встречаться в файлах, закодированных в PEM, в элементах -----BEGIN и -----END (но, как это ни удивительно, не определён репозиторий IANA для этих ключевых слов). Кроме того, заголовочный файл pem.h из пакета OpenSSL (версии 1.0.2d) содержит другие ключевые слова (некоторые из которых явно упразднены в RFC 7468, некоторые могут быть неявно упразднёнными — смотрите примечания ниже). В таблице ниже приведены ключевые слова, собранные из двух источников и снабжённые соответствующими описаниями и комментариями.

Примечание: В закодированных в PEM файлах данные ключевые слова должны присутствовать в обоих элементах BEGIN и END. Для краткости строки -----BEGIN и --END не показаны, то есть, если в таблице приведено ключевое слово CERTIFICATE, то в PEM-файле оно будет присутствовать дважды: в -----BEGIN CERTIFICATE----- и в -----END CERTIFICATE-----.

Ключевое слово	Источник	Применение	Примечания
CERTIFICATE	RFC 7468	Содержит один закодированный в DER сертификат X.509v3, как определено в разделе 4 RFC 5280	В RFC 7468 названы устаревшими ключевые слова X509 CERTIFICATE и X.509 CERTIFICATE
X509 CRL	RFC 7468	Содержит один закодированный в DER список отзыва сертификатов X.509, как определено в разделе 5 RFC 5280	В RFC 7468 названо устаревшим ключевое слово CRL
CERTIFICATE REQUEST	RFC 7468	Содержит один закодированный в DER запрос сертификата PKCS#10 (он же CSR), как определено в RFC 2986 и дополнено в RFC 5967	В RFC 7468 названо устаревшим ключевое слово NEW CERTIFICATE REQUEST

PKCS7	RFC 7468	Содержит закодированный в DER контейнер PKCS#7 (CMS) (в который может быть заключено несколько сертификатов), как определено в RFC 2315	В RFC 7468 названо устаревшим ключевое слово CERTIFICATE CHAIN и неявно осуждается использование нескольких сертификатов в одной структуре PKCS#7 (в RFC 7468 предлагается заменить PKCS#7 на IETF CMS RFC 5652 , смотрите ниже).
CMS	RFC 7468	Содержит закодированный в DER контейнер CMS (в который может быть заключено несколько сертификатов), как определено в RFC 5652	В основном обратно совместим с описанной выше структурой PKCS#7

PRIVATE KEY	RFC 7468	Содержит незашифрованный закодированный в DER контейнер PKCS#8 с одним ключом, как определено в разделе 2 RFC 5958	В RFC 5958 структура PrivateKeyInfo (из RFC 5208) переименована в OneAsymmetricKey, что позволяет помещать в один контейнер как открытый, так и закрытый ключи. Однако, в RFC 7468 этот формат НЕ поддерживается для закодированных в PEM открытых ключей (смотрите описание ключевого слова PUBLIC KEY ниже). Поскольку контейнер PKCS#8 идентифицирует ключевой алгоритм, он неявно делает устаревшим (хотя прямо об этом не говорится) использование ключевых слов RSA PRIVATE KEY, DSA PRIVATE KEY, EC PRIVATE KEY or ANY PRIVATE KEY, DSA PARAMETERS, EC PARAMETERS, DH PARAMETERS (все они могут быть сгенерированы с использованием OpenSSL).
ENCRYPTED PRIVATE KEY	RFC 7468	Содержит зашифрованный закодированный в DER контейнер PKCS#8 с одним ключом, как определено в разделе 3 RFC 5958	

PUBLIC KEY	RFC 7468	Содержит закодированную в DER структуру SubjectPublicKeyInfo (мини-контейнер) с одним открытым ключом, как определено в разделе 4.1.2.7 RFC 5280	Структура SubjectPublicKeyInfo описывает ключевой алгоритм и, следовательно, RFC 7468 неявно делает устаревшим (хотя прямо об этом не говорится) использование ключевых слов DSA PUBLIC KEY, RSA PUBLIC KEY и ECDSA PUBLIC KEY (все они могут быть сгенерированы с использованием OpenSSL).
ATTRIBUTE CERTIFICATE	RFC 7468	Содержит закодированный в DER атрибутный сертификат (Attribute certificate), как определено в RFC 5755	Атрибутные сертификаты представляют собой закодированные в DER сертификаты X.509, НЕ содержащие открытого ключа. В основном они используются в целях авторизации (а не аутентификации). В заголовочном файле pem.h OpenSSL (1.0.2d) нет соответствующего ключевого слова PEM для такого типа сертификата.
CERTIFICATE PAIR	pem.h	??	Определён в заголовочном файле pem.h OpenSSL, но отсутствует в RFC 7468. В настоящий момент целевой контент не установлен.
TRUSTED CERTIFICATE	pem.h	??	Определён в заголовочном файле pem.h OpenSSL, но отсутствует в RFC 7468. В настоящий момент целевой контент не установлен, но предположительно это сертификат, где в атрибуте BasicConstraints атрибут cA установлен в TRUE.

PKCS #7 SIGNED DATA	pem.h	??	Определён в заголовочном файле pem.h OpenSSL, но отсутствует в RFC 7468. В стандарте PKCS#7 разрешено несколько типов контента ('content types'), один из которых — подписанные данные (эта опция не относится к широко внедрённым).
SSL SESSION PARAMETERS	pem.h	??	Определён в заголовочном файле pem.h OpenSSL, но отсутствует в RFC 7468. В настоящий момент целевой контент не установлен.
X9.42 DH PARAMETERS	pem.h	??	Определён в заголовочном файле pem.h OpenSSL, но отсутствует в RFC 7468. В настоящий момент целевой контент не установлен.

Расширения (суффиксы) файлов

Часто расширение (суффикс) файла не слишком помогает в идентификации его содержимого. Если Вам известен контекст применения файла (что в нём должно быть), то следующая информация может помочь определиться с содержимым:

Применение	Расширение файла	Закрытый ключ	Формат	Примечания
------------	---------------------	------------------	--------	------------

Ключ	.pem	да	PEM	Закодированный в PEM ключ в формате DER в контейнере PKCS#8/RFC 5958, может быть зашифрован или не зашифрован (можно понять по ключевому слову PEM), хотя чаще не зашифрован, поскольку так его используют обычно. Ключевые слова PEM: PRIVATE KEY или ENCRYPTED PRIVATE KEY. Версия 1 (0) поддерживает только закрытые ключи, версия 2 (1) поддерживает использование в PKCS#8 как открытых, так и закрытых ключей.
	.der	да	DER	Используется редко. Закрытый ключ в чистом виде, закодированный в DER. Без обёртки или идентификатора алгоритма.

.key	да	PEM	Такое расширение используется во многих *nix-системах для обозначения закрытого ключа. Закодированный в PEM ключ в формате DER в контейнере PKCS#8/RFC 5958, может быть зашифрован или не зашифрован, хотя чаще не зашифрован, поскольку так его используют обычно. Ключевые слова PEM: PRIVATE KEY или ENCRYPTED PRIVATE KEY.
.p8	yes	DER	В RFC 5958 рекомендовано использование этого расширения/суффикса для бинарных (закодированных в DER) файлов PKCS#8. Версия 1 (0) поддерживает только закрытые ключи, версия 2 (1) поддерживает парные открытый и закрытый ключи. Эквивалент меток PEM типов PRIVATE KEY и ENCRYPTED PRIVATE KEY.

Сертификат	.crt	нет	PEM или DER	Обычно в формате PEM (в RFC 7468 предлагается, чтобы это расширение всегда означало PEM-формат). Содержит только сертификат X.509v3. Не контейнер. Такой формат принимают браузеры MSIE, Firefox и Chrome.
	.cer	нет	PEM или DER	Обычно в формате DER (в RFC 7468 предлагается, чтобы это расширение всегда означало DER-формат). Содержит только сертификат X.509v3. Не контейнер. Такой формат принимают браузеры MSIE, Firefox и Chrome.

.pem

может
быть

PEM

Расширение файла не говорит о его содержимом. В файле может быть практически что угодно, от одного ключа до нескольких сертификатов в связке сертификатов УЦ, заключённых в контейнер PKCS#7 (CMS), или даже запрос сертификата PKCS#10. Ключевое слово PEM поможет определиться с содержимым. В некоторых контекстах считается, что файл с таким расширением эквивалентен файлу с расширением .crt. Такой формат принимает только Firefox.

.der	может быть	DER	Расширение файла не говорит о его содержимом. В файле может быть практически что угодно, от одного ключа до нескольких сертификатов в связке сертификатов УЦ, заключённых в контейнер PKCS#7 (CMS), или даже контейнер PKCS#12, или ещё что-то. В некоторых контекстах считается, что файл с таким расширением эквивалентен файлу с расширением .cer. Такой формат принимает только Firefox.
.p12	может быть	PKCS#12 (RFC 7292)	PKCS#12 (RFC 7292) представляет собой формат закодированного в DER контейнера общего назначения. Может содержать (и, как правило, содержит) один или несколько сертификатов X.509v3 и может содержать (и, как правило, содержит) закодированный в DER закрытый ключ, но может содержать и другие типы данных. Такой формат принимают браузеры MSIE и Chrome.

.pfx	да	RFC 7292	PKCS#12 (RFC 7292) представляет собой формат закодированного в DER контейнера общего назначения. То же, что и файл с расширением .p12, но обычно используется в системах Microsoft и по соглашению содержит один (или несколько) сертификатов X.509v3, закодированных в DER, и закодированный в DER закрытый ключ. Такой формат принимают браузеры MSIE и Chrome.
.p7b	нет	PKCS#7 (или RFC 5652)	PKCS#7 (или RFC 5652) CMS DER-контейнер, закодированный в PEM. Может содержать один или несколько сертификатов, а также другие объекты. Такой формат принимают браузеры MSIE, Firefox и Chrome.

Разное	.csr	нет	PEM/PKCS#10	Также может использоваться расширение .pem. Запрос подписи сертификата (CSR). Содержит закодированный в PEM контейнер в формате DER PKCS#10 (RFC 7292), в котором содержатся открытый ключ пользователя, тип алгоритма и атрибуты, которые требуется добавить в сертификат.
	.crl	нет	PKCS#7 или структура списка сертификатов	Список отзыва сертификатов (CRL) представляет собой закодированную в DER структуру списка сертификатов. Может быть заключена в контейнер PKCS#7 (RFC 2315 , или (чаще) просто закодированная в DER структура списка сертификатов, определённая в разделе 5 RFC 5280 . Обычно закодирована в PEM. Такой формат принимают браузеры MSIE и Chrome.

Связки сертификатов

Номинально, ответственность за проверку полученного от сервера сертификата конечного субъекта несёт клиент. При этом всё чаще требуется проверять ещё и промежуточные сертификаты и/или кросс-сертификаты. Поэтому, если раньше сертификаты распространялись по одному, то теперь в практику входит распространение нескольких сертификатов — мультисертификатов (multi-certificate).

Подобные мультисертификаты обычно называются связками сертификатов (certificate bundles), связками сертификатов УЦ (ca-bundles) или цепочками сертификатов (certificate chains). Регулярное обновление миллионов клиентов выдвигает серьёзные требования к логистике, поэтому всё более популярным становится распространение любых промежуточных сертификатов или кросс-сертификатов (и даже корневых сертификатов) в связке с сертификатом сервера во время одного из этапов рукопожатия TLS.

Существует три основных метода создания связок сертификатов:

1. С помощью структуры PKCS#7 (или RFC 5652). Обычно итоговый файл имеет расширение .p7b (поддерживается браузерами MSIE и Chrome для импорта мультисертификатов). Файлы с таким расширением никогда не содержат закрытого ключа.
2. С помощью структуры PKCS#12 (RFC 7292), представляющей собой суперконтейнер для PKCS#7 и PKCS#8. Итоговый файл может иметь расширения .p12 или .pkx (поддерживается браузерами MSIE и Chrome для импорта мультисертификатов). По соглашению, файл с расширением .pkx содержит и сертификат (PKCS#7), и закрытый ключ (PKCS#8); файл с расширением .p12 может как содержать, так и не содержать закрытого ключа. Для веб-серверов IIS требуются файлы с расширением .pfx (хотя также поддерживаются файлы с расширением .p12).
3. Объединение в определенном порядке закодированных в PEM сертификатов. Поскольку PEM-файлы с сертификатами (ключевое слово PEM [CERTIFICATE](#)) представляют собой текстовые файлы, их можно объединить друг с другом вручную с помощью текстового редактора или такой команды unix:

```
cat intermediate2.crt intermediate1.crt root.crt > ca.pem
# Порядок указания сертификатов отражает выполняемую клиентом
# последовательность проверки: сертификат сервера,
# промежуточные/кросс-сертификаты и, наконец, корневой сертификат.
# Полученный в результате файл (в данном случае ca.pem) будет
# использоваться в директиве Apache2 SSLCertificateFile.
```

Подобное склеивание файлов в формате PEM нашло широкое применение из-за его поддержки в Apache. В таких связках сертификатов никогда не присутствуют закрытые ключи.

Команды OpenSSL для конвертации, извлечения и манипуляций с сертификатами и ключами

В этом разделе показаны некоторые команды OpenSSL для выполнения различных манипуляций. В качестве исходного в OpenSSL используется формат PEM.

```
# конвертация PKCS12 > Извлечение сертификата PEM-формате
openssl pkcs12 -clcerts -nokeys -in cert.p12 -out usercert.pem
openssl pkcs12 -clcerts -nokeys -in cert.p12 -out usercert.crt
# извлекается только сертификат

openssl pkcs12 -nocerts -in cert.p12 -out userkey.pem
openssl pkcs12 -nocerts -in cert.p12 -out userkey.key
# извлекается только ключ

# конвертация PEM > PKCS12 (.p12 или .pfx)
# В результате получается файл в DER (бинарном) формате с расширением .p12 или .pfx
openssl pkcs12 -export -out cert.p12 -inkey ./userkey.pem -in ./usercert.pem
openssl pkcs12 -export -out cert.pfx -inkey ./userkey.pem -in ./usercert.pem
#ПРИМЕЧАНИЕ: в обоих приведённых случаях будет запрашиваться парольная фраза, чтобы
её не задавать
# просто нажмите Enter на запрос пароля и его верификации, либо используйте
следующую команду
openssl pkcs12 -export -out cert.p12 -inkey ./userkey.pem -in ./usercert.pem -
nodes -passout pass:
# закодированные в PEM сертификат и ключ конвертируются в формат PKCS#12
(закодирован в DER)
```

Таблица соответствия стандартов PKCS#X и RFC

В этом разделе даны перекрёстные ссылки с широко используемых стандартов PKCS на их RFC-эквиваленты.

Номер PKCS	RFC	Примечание
PKCS#1	RFC 8017	Контейнер для алгоритма RSA, охватывающего криптографические примитивы, схемы шифрования и схемы подписи.
PKCS#5	RFC 8018	Метод парольной защиты зашифрованных данных (используется в PKCS#8, PKCS#7 и PKCS#12).
PKCS#7	RFC 2315	Контейнер синтаксиса криптографического сообщения (Cryptographic Message Syntax, CMS). Обычно закодирован в PEM. Помимо других сущностей, в него часто заключаются один или несколько CRL (в этом случае файл может иметь расширение .crl, но не всегда), или один или несколько сертификатов (ExtendedCertificatesAndCertificates) (расширение файла .p7b). Отдельный стандарт CMS от IETF (в основном, но не полностью, совместимый с PKCS#7) определён в RFC 5652 .

PKCS#8	RFC 5958	RFC переопределяет спецификацию PKCS#8. Контейнер ключей версии 2 (1) поддерживает и закрытый, и открытый ключи, а контейнер версии 1 (0) — только закрытый ключ. Опционально может быть зашифрован (небезопасно, если не зашифрован!). Если закодирован в PEM, в RFC рекомендуется использовать файл с расширением .pem, если закодирован в DER, рекомендуется использовать файл с расширением .p8 (поддерживается не везде).
PKCS#9	RFC 2985 и RFC 7894	Атрибуты расширения, которые могут использоваться в PKCS#7, PKCS#8 и PKCS#10.
PKCS#10	RFC 2986 , обновлено в RFC 5967	Закодированный в DER контейнер запроса подписи сертификата. Содержит несколько атрибутов, описывающих алгоритм открытого ключа, а также атрибуты которые будут включены в итоговый сертификат. Почти всегда в формате PEM. Расширение файла, как правило, .csr.
PKCS#12	RFC 7292	Общий контейнер для персональных данных. Данный контейнер состоит из контейнеров PKCS#7 и PKCS#8 внутри защищённой структуры. Расширения файлов — .p12 и .pfx. Исключительно в формате DER.

Работа с сертификатами в основных браузерах

Сертификаты могут быть импортированы в некоторые системы/браузеры с помощью процедур, описанных в этом разделе. Периодически эта информация может изменяться. Для браузеров приводится версия браузера, чтобы показать, для какой именно версии приведённый метод является актуальным. Описывается импорт корневого или промежуточного сертификатов (в случае Chrome и MSIE должны быть выбраны соответствующие вкладки "Промежуточные центры сертификации" (Intermediate) или "Доверенные корневые центры сертификации" (Trusted Root)). Только Firefox напрямую принимает файлы с расширением .pem. Для MSIE и Chrome любые файлы с корневыми сертификатами, созданные с помощью описанных выше методов 1, 2, 3 или 3А, имеющие расширение .pem, можно просто переименовать из ca/cacert.pem в ca/cacert.cer или ca/cacert.crt по Вашему выбору.

MSIE (11): MSIE принимает файлы с расширением .cer, .crt, .p7b, .pfx, .p12 (возможно, и другие). MSIE -> Меню "Сервис" (Tools) -> "Свойства обозревателя" (Internet Options) -> Вкладка "Содержание" (Content) -> Кнопка "Сертификаты" (Certificates) ->

Выберите соответствующее хранилище сертификатов -> Кнопка "Импорт" (Import), а затем выполните запросы мастера импорта.

Альтернативный метод для систем Windows 7+: использовать Консоль управления (Microsoft Management Console, MMC) с установленной оснасткой "Сертификаты" (Certificate). Перейдите к соответствующему хранилищу сертификатов -> Меню "Действие" (Actions) -> "Все задачи" (All tasks) -> "Импорт" (Import), а затем выполните запросы мастера импорта (принимает файлы с расширением .cer, .crt, .p7b, .pfx, .p12 (возможно, и другие)).

В окружении AD сертификаты также можно распространить с помощью объектов групповых политик (Group Policy Object, GPO). Загрузите "Администрирование" (Administrative Tools) -> "Управление групповой политикой" (Group Policy Manager) -> Раскройте пункт "Домены" (Domains) -> Щёлкните правой кнопкой мыши на "Default Domain Policy" и выберите "Изменить" (Edit) -> Раскройте пункт "Конфигурация компьютера" (Computer configuration) -> Раскройте пункт "Политики" -> Раскройте пункт "Конфигурация Windows" (Windows Settings) -> Раскройте пункт "Параметры безопасности" (Security Settings) -> Раскройте пункт "Политики открытого ключа" (Public Key Settings) -> Щёлкните правой кнопкой мыши на "Доверенные корневые центры сертификации" (Trusted Root Certificate Authorities) -> Выберите "Импорт" (Import), а затем выполните запросы мастера импорта.

Firefox (41.x.x) Firefox принимает файлы с расширением .pem, .cer, .crt, .der или .p7b. Выберите меню "Инструменты" (Tools) -> "Настройки" (Options) -> "Дополнительные" (Advanced) -> Вкладка "Сертификаты" (Certificates) -> "Просмотр сертификатов" (View Certificates) -> Нажмите кнопку "Импортировать" (Import) и выберите файл.

Chrome (46.x.x.x) Chrome принимает файлы с расширением .cer, .crt, .p7b, .pfx, .p12 suffix (возможно, и другие). Выберите "Настройки" (Settings) -> Нажмите ссылку "Показать дополнительные настройки" (Enable Advanced Settings) -> Перейдите к разделу HTTPS/SSL -> Кнопка "Настроить сертификаты" (Manage Certificates) -> Выберите соответствующую вкладку -> Кнопка "Импорт" (Import), а затем выполните запросы мастера импорта.

RFC по теме

Список RFC, относящихся к TLS, сертификатам X.509 и PKI. Большинство из них упоминается в тексте этого документа. Список не полный, но мы обновляем его по мере обновления текста или при публикации подходящего RFC. Так что в конечном счёте он станет исчерпывающим.

RFC 2315	PKCS #7: Cryptographic Message Syntax Version 1.5PKCS #7: Синтаксис криптографического сообщения, версия 1.5B. Kaliski. Март 1998 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC2315. Связанный формат CMS смотрите также в RFC 5652 .
RFC 2585	Internet X.509 Public Key Infrastructure Operational Protocols: FTP and HTTP.Операционные протоколы для Internet X.509 PKI: FTP и HTTPR. Housley, P. Hoffman. Май 1999 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC2585. Определяет список расширений файлов и ожидаемого содержимого этих файлов.
RFC 2986	PKCS #10: Certification Request Syntax Specification Version 1.7PKCS #10: Спецификация синтаксиса запроса сертификации, версия 1.7M. Nystrom, B. Kaliski. Ноябрь 2000 г. Отменяет RFC2314, обновлён в RFC5967, статус: INFORMATIONAL.
RFC 4210	Internet X.509 Public Key Infrastructure Certificate Management Protocol (CMP)Протокол управления сертификатами (CMP) инфраструктуры открытых ключей X.509 ИнтернетC. Adams, S. Farrell, T. Kaese, T. Mononen. Сентябрь 2005 г. Отменяет RFC2510, обновлён в RFC6712, статус: PROPOSED STANDARD.
RFC 4211	Internet X.509 Public Key Infrastructure Certificate Request Message Format (CRMF).Формат сообщения запроса сертификата (CRMF) инфраструктуры открытых ключей X.509 ИнтернетJ. Schaad. Сентябрь 2005 г. Отменяет RFC2511, статус: PROPOSED STANDARD.
RFC 5019	The Lightweight Online Certificate Status Protocol (OCSP) Profile for High-Volume EnvironmentsОблегчённый профиль протокола онлайн-получения статуса сертификата (OCSP) для высоко нагруженных средA. Deacon, R. Hurst. Сентябрь 2007 г. Статус: PROPOSED STANDARD.
RFC 5246	The Transport Layer Security (TLS) Protocol Version 1.2Протокол TLS, версия 1.2T. Dierks, E. Rescorla. Август 2008 г. Отменяет RFC3268, RFC4346, RFC4366, обновляет RFC4492, обновлён в RFC5746, RFC5878, RFC6176, статус: PROPOSED STANDARD.
RFC 5272	Certificate Management over CMS (CMC)Управление сертификатами поверх CMS (CMC)J. Schaad, M. Myers. Июнь 2008 г. Отменяет RFC2797, обновлён в RFC6402, статус: PROPOSED STANDARD.

RFC 5273	Certificate Management over CMS (CMC): Transport ProtocolsУправление сертификатами поверх CMS (CMC): Транспортные протоколыJ. Schaad, M. Myers. Июнь 2008 г. Обновлён в RFC6402, статус: PROPOSED STANDARD.
RFC 5274	Certificate Management Messages over CMS (CMC): Compliance RequirementsУправление сертификатами поверх CMS (CMC): Технические требованияJ. Schaad, M. Myers. Июнь 2008 г. Обновлён в RFC6402, статус: PROPOSED STANDARD.
RFC 5280	Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) ProfileПрофиль сертификата и списка отзыва сертификатов (CRL) Internet X.509 PKID. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, W. Polk. Май 2008 г. Отменяет RFC3280, RFC4325, RFC4630. Обновлён в RFC6818. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC5280.
RFC 5652	Cryptographic Message Syntax (CMS)Синтаксис криптографического сообщения (CMS)R. Housley. Сентябрь 2009 г. Он же STD0070. Отменяет RFC3852. Статус: INTERNET STANDARD. DOI: 10.17487/RFC5652. Почти полностью обратно совместим с PKCS#7 (RFC 2315).
RFC 5746	Transport Layer Security (TLS) Renegotiation Indication ExtensionРасширение индикации повторных переговоров TLSE. Rescorla, M. Ray, S. Dispensa, N. Oskov. Февраль 2010 г. Обновляет RFC5246, RFC4366, RFC4347, RFC4346, RFC2246, статус: PROPOSED STANDARD.
RFC 5878	Transport Layer Security (TLS) Authorization ExtensionsРасширения авторизации TLSM. Brown, R. Housley. Май 2010 г. Обновляет RFC5246, статус: EXPERIMENTAL.
RFC 5911	New ASN.1 Modules for Cryptographic Message Syntax (CMS) and S/MIME.Новые модули ASN.1 для CMS и S/MIMEP. Hoffman, J. Schaad. Июнь 2010 г. Обновлён в RFC6268, статус: INFORMATIONAL. DOI: 10.17487/RFC5911.
RFC 5912	New ASN.1 Modules for the Public Key Infrastructure Using X.509 (PKIX)Новые модули ASN.1 для инфраструктуры открытых ключей с использованием X.509 (PKIX)P. Hoffman, J. Schaad. Июнь 2010 г. Обновлён в RFC6960, статус: INFORMATIONAL.

RFC 5914	Trust Anchor FormatФормат доверенной связкиR. Housley, S. Ashmore, C. Wallace. Июнь 2010 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC5914.
RFC 5937	Using Trust Anchor Constraints during Certification Path ProcessingИспользование ограничений доверенной связки при обработке пути сертификацииS. Ashmore, C. Wallace. Август 2010 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC5937.
RFC 5958	Asymmetric Key PackagesПакеты асимметричных ключейS. Turner. Август 2010 г. Отменяет RFC5208. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC5958. Заменяет PKCS#8.
RFC 5967	The application/pkcs10 Media TypeМедиа тип application/pkcs10S. Turner. Август 2010 г. Обновляет RFC2986, статус: INFORMATIONAL.
RFC 6066	Transport Layer Security (TLS) Extensions: Extension DefinitionsРасширения TLS: определение расширенияD. Eastlake 3rd. Январь 2011 г. Отменяет RFC4366, статус: PROPOSED STANDARD.
RFC 6125	Representation and Verification of Domain-Based Application Service Identity within Internet Public Key Infrastructure Using X.509 (PKIX) Certificates in the Context of Transport Layer Security (TLS)Представление и верификация идентификационной сущности основанного на домене сервиса приложений в PKIX в контексте TLSP. Saint-Andre, J. Hodges. Март 2011 г. Статус: PROPOSED STANDARD.
RFC 6347	Datagram Transport Layer Security Version 1.2Протокол DTLS версии 1.2E. Rescorla, N. Modadugu. Январь 2012 г. Отменяет RFC4347. Обновлён в RFC7507, статус: PROPOSED STANDARD. DOI: 10.17487/RFC6347.
RFC 6176	Prohibiting Secure Sockets Layer (SSL) Version 2.0Запрет SSL версии 2.0S. Turner, T. Polk. Март 2011 г. Обновляет RFC2246, RFC4346, RFC5246, статус: PROPOSED STANDARD.

RFC 6268	Additional New ASN.1 Modules for the Cryptographic Message Syntax (CMS) and the Public Key Infrastructure Using X.509 (PKIX).Новые модули ASN.1 для CMS и PKIXJ. Schaad, S. Turner. Июль 2011 г. Обновляет RFC5911, статус: INFORMATIONAL. DOI: 10.17487/RFC6268.
RFC 6402	Certificate Management over CMS (CMC) UpdatesОбновления CMCJ. Schaad. Ноябрь 2011 г. Обновляет RFC5272, RFC5273, RFC5274, статус: PROPOSED STANDARD.
RFC 6712	Internet X.509 Public Key Infrastructure — HTTP Transfer for the Certificate Management Protocol (CMP)Инфраструктура открытых ключей X.509 Интернет — передача CMP по HTTPТ. Kaese, M. Peylo. Сентябрь 2012 г. Обновляет RFC4210, статус: PROPOSED STANDARD.
RFC 6818	Updates to the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) ProfileОбновления для профиля сертификата и списка отзыва сертификатов (CRL) Internet X.509 PKIP Yee. Январь 2013 г. Обновляет RFC5280. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC6818
RFC 6960	X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSPПротокол OCSP инфраструктуры открытых ключей X.509 ИнтернетS. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, C. Adams. Июнь 2013 г. Отменяет RFC2560, RFC6277, обновляет RFC5912, статус: PROPOSED STANDARD.
RFC 6961	The Transport Layer Security (TLS) Multiple Certificate Status Request ExtensionРасширение TLS запроса статуса нескольких сертификатовY. Pettersen. Июнь 2013 г. Статус: PROPOSED STANDARD.
RFC 6962	Certificate TransparencyПрозрачность сертификатаB. Laurie, A. Langley, E. Kasper. Июнь 2013 г. Статус: EXPERIMENTAL. DOI: 10.17487/RFC6962.
RFC 7027	Elliptic Curve Cryptography (ECC) Brainpool Curves for Transport Layer Security (TLS)Криптография на основе эллиптических кривых (ECC): кривые Brainpool для TLSJ. Merkle, M. Lochter. Октябрь 2013 г. Обновляет RFC4492, статус: INFORMATIONAL.

RFC 7030	Enrollment over Secure TransportРегистрация поверх безопасного транспортаM. Pritikin, Ed., P. Yee, Ed., D. Harkins, Ed.. Октябрь 2013 г. Статус: PROPOSED STANDARD.
RFC 7091	GOST R 34.10-2012: Digital Signature AlgorithmАлгоритм цифровой подписи GOST R 34.10-2012V. Dolmatov, Ed., A. Degtyarev. Декабрь 2013 г. Обновляет RFC5832, статус: INFORMATIONAL.
RFC 7093	Additional Methods for Generating Key Identifiers ValuesДополнительные методы генерации значений идентификаторов ключаS. Turner, S. Kent, J. Manger. Декабрь 2013 г. Статус: INFORMATIONAL.
RFC 7250	Using Raw Public Keys in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS).Использование необработанных открытых ключей в TLS и DTLS P. Wouters, Ed., H. Tschofenig, Ed., J. Gilmore, S. Weiler, T. Kivinen. Июнь 2014. Статус: PROPOSED STANDARD.
RFC 7251	AES-CCM Elliptic Curve Cryptography (ECC) Cipher Suites for TLS.Шифры AES-CCM Elliptic Curve Cryptography (ECC) для TLS D. McGrew, D. Bailey, M. Campagna, R. Dugal. Июнь 2014 г. Статус: INFORMATIONAL.
RFC 7292	PKCS #12: Personal Information Exchange Syntax v1.1PKCS #12: Синтаксис обмена персональными данными v1.1K. Moriarty, Ed., M. Nystrom, S. Parkinson, A. Rusch, M. Scott. Июль 2014 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC7292
RFC 7457	Summarizing Known Attacks on Transport Layer Security (TLS) and Datagram TLS (DTLS)Обзор известных атак на Transport Layer Security (TLS) и Datagram TLS (DTLS)Y. Sheffer, R. Holz, P. Saint-Andre. Февраль 2015 г. Статус: INFORMATIONAL.
RFC 7468	Textual Encodings of PKIX, PKCS, and CMS StructuresТекстовые кодировки структур PKIX, PKCS и CMSS. Josefsson, S. Leonard. Апрель 2015 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7468

RFC 7507	TLS Fallback Signaling Cipher Suite Value (SCSV) for Preventing Protocol Downgrade Attacks.Значение TLS Fallback SCSV для предотвращения атак типа "downgrade attack"В. Moeller, A. Langley. Апрель 2015 г. Обновляет RFC2246, RFC4346, RFC4347, RFC5246, RFC6347. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7507.
RFC 7525 (BCP0195)	Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)Рекомендации по безопасному использованию TLS и DTLSY. Sheffer, R. Holz, P. Saint-Andre. Май 2015 г. Статус: BEST CURRENT PRACTICE. DOI: 10.17487/RFC7525.
RFC 7568	Deprecating Secure Sockets Layer Version 3.0.Отмена SSLv3R. Barnes, M. Thomson, A. Pironti, A. Langley. Июнь 2015 г. Обновляет RFC5246. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7568.
RFC 7627	Transport Layer Security (TLS) Session Hash and Extended Master Secret ExtensionРасширение TLS хэша сессии и Extended Master SecretK. Bhargavan, Ed., A. Delignat-Lavaud, A. Pironti, A. Langley, M. Ray. Сентябрь 2015 г. Обновляет RFC5246. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7627
RFC 7633	X.509v3 Transport Layer Security (TLS) Feature ExtensionРасширение сертификата x.509v3 TLS FeatureP. Hallam-Baker. Октябрь 2015 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7633.
RFC 7670	Generic Raw Public-Key Support for IKEv2Общая поддержка необработанных открытых ключей для IKEv2T. Kivinen, P. Wouters, H. Tschofenig. Январь 2016 г. Обновляет RFC7296, статус: PROPOSED STANDARD. DOI: 10.17487/RFC7670.
RFC 7685	A Transport Layer Security (TLS) ClientHello Padding ExtensionРасширение TLS дополнения сообщения ClientHelloA. Langley. Октябрь 2015 г. Обновляет RFC5246. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7685.
RFC 7711	PKIX over Secure HTTP (POSH)PKIX поверх защищённого HTTP (POSH)M. Miller, P. Saint-Andre. Ноябрь 2015 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7711.

RFC 7773	Authentication Context Certificate ExtensionРасширение сертификата Authentication ContextS. Santesson. Март 2016 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7773.
RFC 7804	Salted Challenge Response HTTP Authentication MechanismМеханизм аутентификации HTTP Salted Challenge ResponseA. Melnikov. Март 2016 г. Статус: EXPERIMENTAL. DOI: 10.17487/RFC7804.
RFC 7817	Updated Transport Layer Security (TLS) Server Identity Check Procedure for Email-Related ProtocolsОбновлённая процедура проверки идентификационной сущности сервера TLS для протоколов, связанных с электронной почтойA. Melnikov. Март 2016 г. Обновляет RFC2595, RFC3207, RFC3501, RFC5804, статус: PROPOSED STANDARD. DOI: 10.17487/RFC7817.
RFC 7894	Alternative Challenge Password Attributes for Enrollment over Secure TransportАльтернативные атрибуты пароля вызова для протокола ESTM. Pritikin, C. Wallace. Июнь 2016 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7894.
RFC 7906	NSA's Cryptographic Message Syntax (CMS) Key Management AttributesАтрибуты управления ключами для NSA CMSP. Timmel, R. Housley, S. Turner. Июнь 2016 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC7906.
RFC 7918	Transport Layer Security (TLS) False Start"Фальстарт" для TLSA. Langley, N. Modadugu, B. Moeller. Август 2016 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC7918.
RFC 7919	Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)Согласованные параметры конечного поля DHE для TLSGillmor. Август 2016 г. Обновляет RFC2246, RFC4346, RFC4492, RFC5246, Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7919.
RFC 7924	Transport Layer Security (TLS) Cached Information ExtensionРасширение TLS информирования о закешированной информацииS. Santesson, H. Tschofenig. Июль 2016 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7924.

RFC 7925	Transport Layer Security (TLS) / Datagram Transport Layer Security (DTLS) Profiles for the Internet of ThingsПрофили TLS/DTLS для Интернета вещейH. Tschofenig, Ed., T. Fossati. Июль 2016 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC7925.
RFC 7935	The Profile for Algorithms and Key Sizes for Use in the Resource Public Key InfrastructureПрофиль алгоритмов и размеров ключей для использования в RPKIG. Huston, G. Michaelson, Ed.. Август 2016 г. Отменяет RFC6485, статус: PROPOSED STANDARD. DOI: 10.17487/RFC7935.
RFC 8017	PKCS #1: RSA Cryptography Specifications Version 2.2PKCS #1: Спецификации криптографии RSA версии 2.2K. Moriarty, Ed., B. Kaliski, J. Jonsson, A. Rusch. Ноябрь 2016 г. Отменяет RFC3447, статус: INFORMATIONAL. DOI: 10.17487/RFC8017.
RFC 8018	PKCS #5: Password-Based Cryptography Specification Version 2.1PKCS #5: Спецификация основанной на паролях криптографии версии 2.1K. Moriarty, Ed., B. Kaliski, A. Rusch. Январь 2017 г. Отменяет RFC2898, статус: INFORMATIONAL. DOI: 10.17487/RFC8018.
RFC 8062	Anonymity Support for KerberosПоддержка анонимности для KerberosL. Zhu, P. Leach, S. Hartman, S. Emery, Ed. Февраль 2017 г. Отменяет RFC6112, обновляет RFC4120, RFC4121, RFC4556, статус: PROPOSED STANDARD. DOI: 10.17487/RFC8062.
RFC 8125	Requirements for Password-Authenticated Key Agreement (PAKE) SchemesТребования для схем PAKEJ. Schmidt. Апрель 2017 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8125.
RFC 8225	PASSporT: Personal Assertion TokenPASSporT: Персональный токен подтвержденияC. Wendt, J. Peterson. Февраль 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8225.
RFC 8226	Secure Telephone Identity Credentials: CertificatesИдентификационные данные для защиты телефонии: сертификатыJ. Peterson, S. Turner. Февраль 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8226.
RFC 8295	EST (Enrollment over Secure Transport) ExtensionsРасширение EST (Регистрация через безопасный транспорт)S. Turner. Январь 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8295.

RFC 8032	Edwards-Curve Digital Signature Algorithm (EdDSA)Алгоритм цифровой подписи на кривых Эдвардса (EdDSA)S. Josefsson, I. Liusvaara. Январь 2017 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8032.
RFC 8314	Cleartext Considered Obsolete: Use of Transport Layer Security (TLS) for Email Submission and AccessПризнание устаревшим использования открытого текста: применение TLS для отправки и доступа к электронной почтеK. Moore, C. Newman. Январь 2018 г. Обновляет RFC1939, RFC2595, RFC3501, RFC5068, RFC6186, RFC6409. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8314.
RFC 8351	The PKCS #8 EncryptedPrivateKeyInfo Media TypeМедиа тип для PKCS #8 EncryptedPrivateKeyInfoS. Leonard. Июнь 2018 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8351.
RFC 8360	Resource Public Key Infrastructure (RPKI) Validation ReconsideredПересмотр валидации RPKIG. Huston, G. Michaelson, C. Martinez, T. Bruijnzeels, A. Newton, D. Shaw. Апрель 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8360.
RFC 8398	Internationalized Email Addresses in X.509 CertificatesИнтернационализированные адреса электронной почты в сертификатах X.509Под редакцией A. Melnikov, W. Chuang. Май 2018 г. Обновляет RFC5280. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8398.
RFC 8399	Internationalization Updates to RFC 5280Касающиеся интернационализации обновления RFC 5280R. Housley. Май 2018 г. Обновляет RFC5280. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8399.
RFC 8410	Algorithm Identifiers for Ed25519, Ed448, X25519, and X448 for Use in the Internet X.509 Public Key InfrastructureИдентификаторы алгоритмов Ed25519, Ed448, X25519 и X448 для использования в Интернет PKI X.509S. Josefsson, J. Schaad. Август 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8410.
RFC 8411	IANA Registration for the Cryptographic Algorithm Object Identifier Range.Регистрация IANA для диапазона идентификаторов объектов криптографических алгоритмовJ. Schaad, R. Andrews. Август 2018 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8411.

RFC 8418	Use of the Elliptic Curve Diffie-Hellman Key Agreement Algorithm with X25519 and X448 in the Cryptographic Message Syntax (CMS)Использование алгоритма согласования ключей Диффи-Хеллмана на эллиптических кривых с X25519 и X448 в CMSR. Housley. Август 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8418.
RFC 8419	Use of Edwards-Curve Digital Signature Algorithm (EdDSA) Signatures in the Cryptographic Message Syntax (CMS)Использование подписей на основе алгоритма EdDSA в CMSR. Housley. Август 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8419.
RFC 8422	Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS) Versions 1.2 and EarlierШифры на основе эллиптических кривых для TLS 1.2 и более ранних версийY. Nir, S. Josefsson, M. Pegourie-Gonnard. Август 2018 г. Отменяет RFC4492. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8422.
RFC 8423	Reclassification of Suite B Documents to Historic StatusПереквалификация документов Suite B в статус историческихR. Housley, L. Ziegler. Июль 2018 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8423.
RFC 8442	ECDHE_PSK with AES-GCM and AES-CCM Cipher Suites for TLS 1.2 and DTLS 1.2Шифры ECDHE_PSK с AES-GCM и AES-CCM для TLS 1.2 и DTLS 1.2J. Mattsson, D. Migault. Сентябрь 2018 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8442.
RFC 8446	The Transport Layer Security (TLS) Protocol Version 1.3Протокол TLS версии 1.3E. Rescorla. Август 2018 г. Отменяет RFC5077, RFC5246, RFC6961. Обновляет RFC5705, RFC6066. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8446.
RFC 8447	IANA Registry Updates for TLS and DTLSОбновление реестра IANA для TLS и DTLSJ. Salowey, S. Turner. Август 2018 г. Обновляет RFC3749, RFC4680, RFC5077, RFC5246, RFC5705, RFC5878, RFC6520, RFC7301. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8447.

RFC 8448	Example Handshake Traces for TLS 1.3Примеры выполнения протокола рукопожатия в TLS 1.3М. Thomson. Январь 2019 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8448.
RFC 8449	Record Size Limit Extension for TLSPасширение TLS Предел размера записиМ. Thomson. Август 2018 г. Обновляет RFC6066. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8449.
RFC 8452	AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption.AES-GCM-SIV: Аутентифицированное шифрование, устойчивое к неверному использованию отметки времениS. Gueron, A. Langley, Y. Lindell. Апрель 2019 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8452.
RFC 8479	Storing Validation Parameters in PKCS#8Хранение параметров валидации в PKCS#8N. Mavrogiannopoulos. Сентябрь 2018 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8479.
RFC 8492	Secure Password Ciphersuites for Transport Layer Security (TLS)Шифры для TLS на основе безопасных паролейПо редакцией D. Harkins. Февраль 2019 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8492.
RFC 8554	Leighton-Micali Hash-Based Signatures.Подписи на основе хеша по алгоритмам Лейтона-Микали.D. McGrew, M. Curcio, S. Fluhrer. Апрель 2019 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8554.
RFC 8555	Automatic Certificate Management Environment (ACME)Среда автоматического управления сертификатами (ACME)R. Barnes, J. Hoffman-Andrews, D. McCarney, J. Kasten. Март 2019 г. Статус: PROPOSED STANDARD. DOI: 10.17487/RFC8555.
RFC 8603	Commercial National Security Algorithm (CNSA) Suite Certificate and Certificate Revocation List (CRL) Profile.Профиль сертификата и CRL для CNSAM. Jenkins, L. Ziegler. Май 2019 г. Статус: INFORMATIONAL. DOI: 10.17487/RFC8603.

Изменения страницы

Дата последнего изменения указана внизу страницы.

1. 5 декабря 2019 г.: Страница реструктурирована: примеры генерации сертификатов и CSR вынесены на отдельную страницу (для уменьшения объёма). Изменена нумерация примеров, добавлен пример с генерацией CSR, обновлён номер версии OpenSSL — до 1.1.1d.
2. 4 июня 2019 г.: Добавлено примечание об усилении стойкости ключа в сертификатах X.509 для CNSA. Обновлён список RFC.
3. 26 марта 2019 г.: Добавлено примечание об использовании паролей (вместо сертификатов) для аутентификации в TLS, примечание по протоколу ACME, а также о свободных сертификатах, выпуск которых управляется ACME.
4. 22 марта 2019 г.: Обновлён список RFC.
5. 6 сентября 2018 г.: Иллюстрация и описание возобновления сессии TLS 1.3. В иллюстрацию полного протокола рукопожатия TLS 1.3 добавлено сообщение NewSessionTicket.
6. 5 сентября 2018 г.: Иллюстрация и описание полного протокола рукопожатия TLS 1.3.
7. 15 августа 2018 г.: Существенные изменения, касающиеся принятия TLS 1.3. Представление расширения "record_size_limit" (TLS 1.2+). Обновлён список RFC.
8. 8 августа 2018 г.: Обновлён список RFC.
9. 1 августа 2018 г.: Замена TLS Suite B на CNSA Suite (RFC 8423). Добавлено дополнительное примечание относительно имён в сертификатах X.509. Обновлён список RFC.
10. 18 апреля 2018 г.: Дополнительные примечания по наборам шифров. Даны ссылки на реестр IANA и страницу с описанием манипуляций с наборами шифров.
11. 11 октября 2017 г.: Добавлено текстовое пояснение в определении Удостоверяющего центра.
12. 14 сентября 2017 г.: Добавлено расширение (суффикс) файла .p8. Уточнено, что PKCS v2 поддерживает и закрытый и открытый ключи, v1 — только закрытый. Исправлены примечания по расширению файла .key.
13. 2 сентября 2017 г.: В содержании исправлена ссылка на журнал изменений. В некоторых расширениях X.509 V3 для указания значений контекстно-зависимых тегов (TaggedTypes) используются круглые, а не квадратные скобки. Добавлен ASN.1-фрагмент X.509. Добавлено несколько ссылок. Уточнено, что в методе 2 в качестве корневого и серверного используется один и тот же сертификат. Исправлены опечатки.
14. 23 августа 2017 г.: Незначительная корректировка и исправление опечаток.
15. 12 мая 2017 г.: Добавлено ещё немного увлекательной терминологии, используемой поставщиками сертификатов для описания своих продуктов.

16. 20 апреля 2017 г.: При описании элементов X.509 слово "поле" заменено технически корректным термином "атрибут". Первоначальный замысел использования более общего слова "поле" состоял в том, чтобы уменьшить сложность путём минимизации экзотической терминологии. Но некоторые читатели отметили, что это может привести к путанице, особенно при чтении документации УЦ, в которой упоминаются атрибуты.
17. 20 апреля 2017 г.: Добавлены RFC 8062 и 8125.
18. 25 января 2017 г.: В таблице соответствия стандартов PKCS#X и RFC обновлена ссылка на RFC для PKCS#5, добавлена информация по PKCS#1 со ссылкой на RFC. В список RFC добавлены RFC 8017 и 8018.
19. 16 сентября 2016: Исправлена опечатка в аббревиатуре CRMF. Добавлено примечание о введённом в RFC 7918 "фальстарте", позволяющем клиенту начинать передачу сообщения сразу после отправки 'Finished', не дожидаясь получения 'Finished' от сервера. В список RFC добавлены RFC 7918 и 7935.
20. 28 июля 2016 г.: Обновлён список RFC. Новый раздел об использовании полей subject и subjectAltName в сертификатах.
21. 13 июля 2016 г.: Добавлены недостающие гиперссылки.
22. 13 февраля 2016 г.: Обновлён список RFC, исправлены некорректные гиперссылки. Новый раздел об использовании сертификатов в окружении (многопользовательском) хостинг-провайдеров.
23. 2 ноября 2015 г.: Добавлена информация по связанным с TLS/SSL/сертификатами именам файлов, форматам и PKCS-контейнерам. Обновлена информация по импорту сертификатов в Chrome, Windows, MSIE и Firefox.
24. 28 октября 2015 г.: Обновлён список RFC. Добавлено определение сертификата конечного субъекта.
25. 23 октября 2015 г.: Обновлён список RFC. Добавлено примечание по расширению, позволяющему дополнять нулями размер сообщения ClientHello. Добавлено примечание по определённому в RFC 7633 расширению типа "Private Internet", позволяющее включать в сертификат расширения функционала TLS, что помогает в предотвращении атак. В описание расширений сертификата добавлены OID, чтобы можно было различить стандартные расширения X.509 (из пространства 2.5.29) и расширения "Private Internet" (из пространства 1.3.6.1.5.5.7.1). Текст примеров сокращён (в интересах пользователей с небольшими экранами).
26. 29 сентября 2015 г.: Обновлён список RFC. В описание протоколов TLS/SSL добавлено примечание по Extended Master Secret.
27. 14 июля 2015 г.: Обновлён список RFC. Добавлено примечание об отмене SSL v3.0.
28. 10 июля 2015 г.: Обновлён список RFC. Добавлено примечание о сертификатах Data Transmission Content Protection (DTCP) и их использовании в протоколе рукопожатия TLS (как определено в RFC 7562).

29. 30 мая 2015 г.: Обновлён список RFC. Добавлено примечание об определённом в RFC 7507 "наборе шифров" TLS_FALLBACK_SCSV.
30. 15 марта 2015 г.: Обновлён список RFC.
31. 5 января 2015 г.: Исправлена битая ссылка на раздел "Цепочки сертификатов X.509", исправлено неверное указание поля subjectPublicKeyInfo. Исправлена опечатка в описании поля subjectAltName для правильной идентификации использования атрибута dNSName.
32. 4 июля 2014 г.: Примечания об упрощённом формате сертификата, определённом в RFC 7250, в описаниях сообщений ClientHello, ServerHello, формата сертификата X.509 и атрибута SubjectPublicKeyInfo. Обновлён список RFC.
33. 21 января 2014 г.: Примечание в описании сообщения ClientHello о расширении Server Name Indication (SNI). Обновлён список RFC.
34. 22 декабря 2013 г.: Обновлён список RFC.
35. 18 декабря 2013 г.: Обновлён список RFC.
36. Ноябрь 2013 г.: Страница переделана в HTML5
37. Ноябрь 2013 г.: Порядок терминов на странице изменён с SSL/TLS на TLS/SSL для отображения современных тенденций.
38. Ноябрь 2013 г.: Обновлён рисунок 2.
39. Октябрь 2013 г.: В список RFC добавлены RFC 7027 и 7030. Обновлён текст, касающийся ECC.

Проблемы, комментарии, предположения, исправления (включая битые ссылки) или есть что добавить? Пожалуйста, выкроите время в потоке занятой жизни, чтобы [написать нам](#), [вебмастеру](#) или в [службу поддержки](#). Оставшийся день Вы проведёте с чувством удовлетворения.

Нашли ошибку в переводе? [Сообщите переводчикам!](#)

Copyright © 1994-2020 ZyTrax, Inc. Все права защищены. Последнее изменение страницы: 7 декабря 2019 г. Переведено участниками проекта [Pro-LDAP.ru](#) в 2013-2020 г.